Studio Nx

S1-06 타임라인 — 함수 설명 합본 (JS 115~148)

2026-10-15

목차

1. ai-notes/javascript/S1-06 - 타임라인/115_sequencer-arrange_ticksToBars.md
2. ai-notes/javascript/S1-06 - 타임라인/116_sequencer-arrange_barsToTicks.md
3. ai-notes/javascript/S1-06 - 타임라인/117_sequencer-arrange_divisionTicks.md
4. ai-notes/javascript/S1-06 - 타임라인/118_sequencer-arrange_tripletTicks.md
5. ai-notes/javascript/S1-06 - 타임라인/119_sequencer-arrange_snapStepTicks.md
6. ai-notes/javascript/S1-06 - 타임라인/120_sequencer-arrange_snapTick.md
7. ai-notes/javascript/S1-06 - 타임라인/121_sequencer-arrange_moveClipStart.md
8. ai-notes/javascript/S1-06 - 타임라인/122_sequencer-arrange_resizeClipLength.md
9. ai-notes/javascript/S1-06 - 타임라인/123_sequencer-arrange_trimClipLeft.md
10. ai-notes/javascript/S1-06 - 타임라인/124_sequencer-arrange_splitClip.md
11. ai-notes/javascript/S1-06 - 타임라인/125_sequencer-arrange_duplicateClipStart.md
12. ai-notes/javascript/S1-06 - 타임라인/126_sequencer-arrange_clampZoom.md
13. ai-notes/javascript/S1-06 - 타임라인/127_sequencer-arrange_zoomScrollLeft.md
14. ai-notes/javascript/S1-06 - 타임라인/128_sequencer-arrange_stepTrackHeight.md
15. ai-notes/javascript/S1-06 - 타임라인/129_sequencer-arrange_nextTrackNumber.md
16. ai-notes/javascript/S1-06 - 타임라인/130_sequencer-arrange_renumberMap.md
17. ai-notes/javascript/S1-06 - 타임라인/131_sequencer-arrange_reorderItems.md
18. ai-notes/javascript/S1-06 - 타임라인/132_sequencer-arrange_trackColor.md
19. ai-notes/javascript/S1-06 - 타임라인/133_sequencer-arrange_waveformColumns.md
20. ai-notes/javascript/S1-06 - 타임라인/134_sequencer-arrange_waveformBars.md
21. ai-notes/javascript/S1-06 - 타임라인/135_sequencer-arrange_drawWaveform.md
22. ai-notes/javascript/S1-06 - 타임라인/136_sequencer-arrange_snapOptions.md
23. ai-notes/javascript/S1-06 - 타임라인/137_sequencer-arrange_snapDivision.md
24. ai-notes/javascript/S1-06 - 타임라인/138_sequencer-arrange_divisionBars.md
25. ai-notes/javascript/S1-06 - 타임라인/139_sequencer-arrange_snapUnitNow.md
26. ai-notes/javascript/S1-06 - 타임라인/140_sequencer-arrange_snapMenu.md
27. ai-notes/javascript/S1-06 - 타임라인/141_sequencer-arrange_deleteTrack.md
28. ai-notes/javascript/S1-06 - 타임라인/142_sequencer-arrange_trackReorder.md
29. ai-notes/javascript/S1-06 - 타임라인/143_sequencer-arrange_clipResizeRight.md
30. ai-notes/javascript/S1-06 - 타임라인/144_sequencer-arrange_clipTrimLeft.md
31. ai-notes/javascript/S1-06 - 타임라인/145_sequencer-arrange_clipDuplicate.md
32. ai-notes/javascript/S1-06 - 타임라인/146_sequencer-arrange_clipSlice.md
33. ai-notes/javascript/S1-06 - 타임라인/147_sequencer-arrange_zoom.md
34. ai-notes/javascript/S1-06 - 타임라인/148_sequencer-arrange_waveform.md

JS 115 — `ticksToBars()` (tick을 마디로 바꾸기)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js` (편곡 영역 모듈에 추가)
작업	S1-06 타임라인: 첫 번째 함수 (FR-209 좌표 기준)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline` (`develop`에서 만듦)
다음	이 함수의 역방향 `barsToTicks()`와 스냅 단위 함수를 한 함수씩 이어서 진행합니다

1. 하는 일

엔진·DB의 **tick**(1박 = 960)을 편곡 화면의 **마디**로 바꿉니다. 편곡 화면은 마디 단위로 그리고, 소리 엔진과 DB는 tick 단위를 쓰기 때문에 두 단위를 잇는 변환이 필요합니다.

> 마디 = tick ÷ (한 마디의 tick 수) 예: 3840 tick ÷ 3840 = 1마디

입력	결과
0 tick	0마디
960 tick (1박)	0.25마디
480 tick (1/8박)	0.125마디
3840 tick (4/4 한 마디)	1마디
15360 tick (4마디)	4마디
-960 tick (카운트인처럼 0보다 앞)	-0.25마디
두 번째 인자 4800 tick (5/4 한 마디)	1마디
tick이 NaN·Infinity·문자열, 마디 길이가 0 이하	오류(`RangeError`)

2. 코드

```
import { PPQ } from '../audio/timing.js';

export const TICKS_PER_BAR = PPQ * 4;

export function ticksToBars(tick, ticksPerBar = TICKS_PER_BAR) {
  if (typeof tick !== 'number' || !isFinite(tick)) throw new RangeError('tick 은 유한한 숫자여야 합니다:
' + tick);
  if (typeof ticksPerBar !== 'number' || !isFinite(ticksPerBar) || ticksPerBar <= 0) throw new RangeErro
r('ticksPerBar 는 0 보다 큰 숫자여야 합니다: ' + ticksPerBar);
  return tick / ticksPerBar;
}
```

최신 문법으로 쓰면 (공부용)

```
export function ticksToBars(tick, ticksPerBar = TICKS_PER_BAR) {
  if (!Number.isFinite(tick)) throw new RangeError(`tick 은 유한한 숫자여야 합니다: ${tick}`);
  if (!Number.isFinite(ticksPerBar) || ticksPerBar <= 0) throw new RangeError(`ticksPerBar 는 0 보다 큰 숫자여야 합니다: ${ticksPerBar}`);
  return tick / ticksPerBar;
}
```

최신 문법	실제 코드	뜻
`Number.isFinite(x)`	`typeof x !== 'number' || !isFinite(x)`	숫자이면서 NaN·Infinity가 아닌지 확인합니다
템플릿 문자열 `` `…${x}` ``	`'…' + x`	글자와 값을 잇습니다
기본값 인자	그대로 사용	두 번째 인자를 안 주면 4/4(3840)를 씁니다

3. 왜 이렇게

선택	이유
tick → 마디 변환을 먼저 만듦	편곡 화면은 마디, 엔진·DB는 tick을 씁니다. S1-05 재생선 위치(`Transport.positionTick()`)를 마디로 바꿔 넘길 때 가장 먼저 필요합니다
`PPQ`를 `audio/timing.js`에서 가져옴	960이라는 값이 여러 파일에 흩어지지 않게 한 곳만 고칩니다 (S1-05와 같은 출처)
한 마디 tick 수를 인자로도 받음	박자표가 바뀌는 곡(FR-202a)에서는 마디 길이가 달라집니다. 기본값 4/4(3840)로 두고, 필요할 때 인자로 넘깁니다
템포(BPM)는 쓰지 않음	위치만 바꾸는 계산입니다. tick ↔ 초 변환은 `audio/timing.js`가 맡아 역할을 나눕니다
소수 결과를 그대로 돌려줌	반올림·스냅은 부르는 쪽(드래그 코드)이 합니다. 여기서 반올림하면 위치가 미리 틀어집니다
잘못된 값은 오류를 던짐	조용히 NaN을 넘기면 재생선이 엉뚱한 곳에 섭니다

4. 시험 (Node에서 직접 실행, 16개 통과)

확인	결과
0·960·480·3840·15360 tick이 0·0.25·0.125·1·4마디	✓
카운트인(-960 tick)이 -0.25마디	✓
5/4(4800)·3/4(2880) 마디 인자에서 1마디	✓
`TICKS_PER_BAR`가 3840	✓
tick이 NaN·Infinity·없음·문자열이면 오류	✓
마디 길이가 0·음수·NaN이면 오류	✓

JS 116 — `barsToTicks()` (마디를 tick으로 바꾸기)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 두 번째 함수 (FR-208 클립 위치)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	스냅 단위 함수(FR-206)를 이어서 진행합니다

1. 하는 일

편곡 화면의 **마디**를 엔진·DB의 **tick**(1박 = 960)으로 바꿉니다. `ticksToBars()`의 역방향입니다. 클립을 끌어 옮긴 위치(마디)를 `clip.start_tick`처럼 tick으로 저장하거나 엔진에 넘길 때 씁니다.

> tick = 마디 × (한 마디의 tick 수) 예: 1마디 × 3840 = 3840 tick

입력	결과
0마디	0 tick
0.25마디 (1박)	960 tick
0.125마디 (1/8박)	480 tick
1마디 (4/4)	3840 tick
4마디	15360 tick
-0.25마디 (카운트인)	-960 tick
두 번째 인자 4800 tick (5/4 한 마디)	4800 tick
마디가 NaN·Infinity·문자열, 마디 길이가 0 이하	오류(`RangeError`)

2. 코드

```
export function barsToTicks(bar, ticksPerBar = TICKS_PER_BAR) {
  if (typeof bar !== 'number' || !isFinite(bar)) throw new RangeError('bar 는 유한한 숫자여야 합니다: ' + bar);
  if (typeof ticksPerBar !== 'number' || !isFinite(ticksPerBar) || ticksPerBar <= 0) throw new RangeError('ticksPerBar 는 0 보다 큰 숫자여야 합니다: ' + ticksPerBar);
  return bar * ticksPerBar;
}
```

최신 문법으로 쓰면 (공부용)

```
export const barsToTicks = (bar, ticksPerBar = TICKS_PER_BAR) => {
  if (!Number.isFinite(bar)) throw new RangeError(`bar 는 유한한 숫자여야 합니다: ${bar}`);
  if (!Number.isFinite(ticksPerBar) || ticksPerBar <= 0) throw new RangeError(`ticksPerBar 는 0 보다 큰 숫자여야 합니다: ${ticksPerBar}`);
```

```
  return bar * ticksPerBar;
};
```

최신 문법	실제 코드	뜻
`Number.isFinite(x)`	`typeof x !== 'number' || !isFinite(x)`	숫자이면서 NaN·Infinity가 아닌지 확인합니다
템플릿 문자열	`'…' + x`	글자와 값을 잇습니다
화살표 함수	`function` 선언	같은 동작입니다

3. 왜 이렇게

선택	이유
`ticksToBars()`와 짝으로 만듦	화면에서 만든 마디를 저장·엔진용 tick으로 되돌리는 길이 있어야 합니다
`TICKS_PER_BAR`, 검사 규칙을 그대로 씀	두 함수가 같은 기준(4/4 = 3840, 잘못된 값은 오류)을 써서 어긋나지 않습니다
소수 결과를 그대로 돌려줌	반올림은 부르는 쪽이 합니다. 여기서 반올림하면 저장 위치가 미리 틀어집니다
잘못된 값은 오류를 던짐	엉뚱한 tick이 DB에 저장되는 것을 막습니다

4. 시험 (Node에서 직접 실행, 18개 통과)

확인	결과
0·0.25·0.125·1·4마디가 0·960·480·3840·15360 tick	✓
카운트인(-0.25마디)이 -960 tick	✓
5/4(4800)·3/4(2880) 인자에서 1마디	✓
`ticksToBars(barsToTicks(x))` 왕복이 원래 값	✓
마디가 NaN·Infinity·없음·문자열이면 오류	✓
마디 길이가 0·음수·NaN이면 오류	✓

JS 117 — `divisionTicks()` (스냅 칸 이름을 tick 길이로)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 세 번째 함수 (FR-206 그리드 스냅)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	셋잇단(1/n 의 3등분) 칸 길이 함수, 그리고 Snap 메뉴 연결을 이어서 진행합니다

1. 하는 일

상단 Snap 메뉴에서 고르는 **칸 이름**(`1/1` ~ `1/32`)을 그 칸 하나의 **tick 길이**로 바꿉니다. FR-206의 그리드 스냅에서 "몇 tick마다 붙을지"를 정하는 값입니다.

> 1/n 음표의 tick = PPQ(960) × 4 ÷ n (음표 길이라 박자표와 무관합니다)

칸	한 칸 tick	마디로(4/4)
`1/1` (온음표)	3840	1마디
`1/2` (2분음표)	1920	0.5마디
`1/4` (4분음표)	960	0.25마디
`1/8`	480	0.125마디
`1/16`	240	0.0625마디
`1/32`	120	0.03125마디
목록에 없는 이름(`1/3`, `1/64`, 숫자 등)	오류(`RangeError`)	—

2. 코드

```
const DIVISION_DEN = { '1/1': 1, '1/2': 2, '1/4': 4, '1/8': 8, '1/16': 16, '1/32': 32 };

export function divisionTicks(division) {
  const den = DIVISION_DEN[division];
  if (typeof den !== 'number') throw new RangeError('알 수 없는 스냅 칸입니다: ' + division);
  return (PPQ * 4) / den;
}
```

최신 문법으로 쓰면 (공부용)

```
const DIVISION_DEN = { '1/1': 1, '1/2': 2, '1/4': 4, '1/8': 8, '1/16': 16, '1/32': 32 };
export function divisionTicks(division) {
  const den = DIVISION_DEN[division];
  if (!Number.isInteger(den)) throw new RangeError(`알 수 없는 스냅 칸입니다: ${division}`);
  return (PPQ * 4) / den;
}
```

최신 문법	실제 코드	뜻
`Number.isInteger(x)`	`typeof x !== 'number'`	값이 정수인지 확인합니다. 아래 3번처럼 상속된 키를 걸러 내는 데도 씁니다
템플릿 문자열	`'…' + x`	글자와 값을 잇습니다

3. 왜 이렇게

선택	이유
음표 길이를 tick으로 돌려줌	음표 길이는 박자표와 무관합니다(`1/4`는 4/4·3/4 모두 1박 = 960). 마디로 바꾸는 일은 `ticksToBars()`가 맡아 역할을 나눕니다
표(객체)로 분모를 둠	960÷n을 매번 계산하지 않고, 허용된 칸만 받도록 합니다. 화면 Snap 목록과 같은 값을 씁니다
`typeof den !== 'number'`로 검사	`division in 표`나 `표[division]`만 보면 `toString`·`constructor` 같은 **상속된 키**가 통과합니다. 숫자일 때만 돌려주어 막습니다
잘못된 값은 오류를 던짐	없는 칸으로 스냅하면 위치가 조용히 틀어집니다. 바로 알리는 쪽이 낫습니다
셋잇단은 다음 함수	셋잇단 칸은 "1/n을 3등분"이라 규칙이 따로입니다. 한 함수씩 나눠 검토합니다

4. 시험 (Node에서 직접 실행, 15개 통과)

확인	결과
`1/1`~`1/32`가 3840·1920·960·480·240·120 tick	✓
`1/4`·`1/16`을 마디로 바꾸면 0.25·0.0625	✓
없는 칸(`1/3`·`1/64`·빈 문자열), 숫자·없음 → 오류	✓
상속된 키(`toString`·`constructor`) → 오류	✓

JS 118 — `tripletTicks()` (셋잇단 칸의 tick 길이)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 네 번째 함수 (FR-206 셋잇단 포함)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	스냅 단위를 고르는 함수(칸 + 셋잇단 → tick)와 Snap 메뉴 연결을 이어서 진행합니다

1. 하는 일

`³ Triplet`을 켰을 때 그 칸의 **셋잇단 한 칸 길이**를 tick으로 돌려줍니다. 셋잇단은 **한 칸을 3등분**한 길이입니다 (원래 칸 ÷ 3).

> 셋잇단 한 칸 = (같은 이름 칸의 tick) ÷ 3

칸	원래 칸	셋잇단 한 칸
`1/1`	3840	1280
`1/2`	1920	640
`1/4`	960	320
`1/8`	480	160
`1/16`	240	80
`1/32`	120	40
이름이 틀림(`1/3` 등)	—	오류(`RangeError`)

그림으로 보면 `1/4` 칸(960)이 셋잇단일 때 3토막으로 나뉩니다.

```
1/4 칸 (960)
├──────────────┤
├────┼────┼────┤   셋잇단 한 칸 = 320
```

2. 코드

```
export function tripletTicks(division) {
  return divisionTicks(division) / 3;
}
```

최신 문법으로 쓰면 (공부용)

```
export const tripletTicks = (division) => divisionTicks(division) / 3;
```

최신 문법	실제 코드	뜻
화살표 함수	`function` 선언	같은 동작입니다

3. 왜 이렇게

선택	이유
셋잇단 = 원래 칸 ÷ 3	2026-10-14 사용자 결정(그림 (나)). 화면 칸 하나가 그대로 3토막 나는 방식이라 직관적입니다
`divisionTicks()`를 그대로 씀	칸 이름 검사와 값이 한 곳에서 나옵니다. 없는 이름은 `divisionTicks()`가 던진 오류가 그대로 전해집니다
3으로 나눔을 함수로 분리	셋잇단 규칙(2/3 vs 1/3)이 바뀌어도 고칠 곳이 여기 한 곳입니다
표준(× 2/3)을 안 씀	표준은 셋잇단 칸을 원래 칸보다 길게(3개가 2칸) 잡습니다. 사용자가 (나)를 골랐습니다

> 참고: 기존 프로토타입은 '박' 단계가 ÷3, '1/16' 단계가 ×2/3로 서로 달랐습니다. 이 함수로 **모든 칸을 ÷3으로 통일**합니다.

4. 시험 (Node에서 직접 실행, 14개 통과)

확인	결과
`1/1` ~`1/32` 셋잇단이 1280·640·320·160·80·40 tick	✓
셋잇단 3개가 원래 한 칸과 같음(`1/4`, `1/16`)	✓
셋잇단이 원래 칸의 1/3	✓
없는 칸·빈 문자열·숫자·없음·상속 키 → 오류	✓

JS 119 — `snapStepTicks()` (스냅 한 칸 길이 고르기)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 다섯 번째 함수 (FR-206 그리드 스냅)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	tick을 격자에 맞추는 함수(가장 가까운 칸으로)와 Snap 메뉴 연결을 이어서 진행합니다

1. 하는 일

Snap 메뉴에서 고른 **칸**과 `³ Triplet` 여부를 **한 칸의 tick 길이** 하나로 정리합니다. 끌기 코드는 이 값을 스냅 단위로 씁니다.

> 셋잇단이 꺼져 있으면 `divisionTicks()`(예: `1/16` = 240), 켜져 있으면 `tripletTicks()`(예: `1/16` = 80)

칸	보통	셋잇단
`1/1`	3840	1280
`1/2`	1920	640
`1/4`	960	320
`1/8`	480	160
`1/16`	240	80
`1/32`	120	40
이름이 틀림(`1/3` 등)	오류(`RangeError`)	오류(`RangeError`)

2. 코드

```
export function snapStepTicks(division, triplet = false) {
  return triplet ? tripletTicks(division) : divisionTicks(division);
}
```

최신 문법으로 쓰면 (공부용)

```
export const snapStepTicks = (division, triplet = false) =>
  triplet ? tripletTicks(division) : divisionTicks(division);
```

최신 문법	실제 코드	뜻
화살표 함수	`function` 선언	같은 동작입니다
기본값 인자	그대로 사용	`triplet`을 안 주면 보통(꺼짐)으로 봅니다

3. 왜 이렇게

선택	이유
칸 + 셋잇단을 한 함수로	끌기 코드가 매번 "셋잇단이면 어느 값"을 따지지 않고 이 함수 하나만 부르면 됩니다
두 값 함수를 그대로 씀	칸 값(`divisionTicks`)과 셋잇단 값(`tripletTicks`)이 각각 한 곳에서 나옵니다. 검사·오류도 그대로 전해집니다
`triplet` 기본값 false	두 번째 인자를 안 주는 곳(Snap Off·Line 등)에서도 안전하게 보통 칸을 돌려줍니다
없는 칸은 오류	조용히 0이나 NaN을 돌려주면 스냅이 꺼진 것처럼 보입니다. 바로 알리는 쪽이 낫습니다

4. 시험 (Node에서 직접 실행, 15개 통과)

확인	결과
`1/4`·`1/8`·`1/16` 보통/셋잇단이 960/320·480/160·240/80	✓
`1/1` 보통 3840, `1/32` 셋잇단 40	✓
`triplet` 생략·false는 보통, true는 `tripletTicks()`와 같음	✓
없는 칸(보통·셋잇단)·숫자·없음 → 오류	✓

JS 120 — `snapTick()` (가장 가까운 격자로 맞추기)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 여섯 번째 함수 (FR-206 그리드 스냅 동작)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	클립 위치·길이를 tick으로 다루는 함수(이동·크기 조절)와 Snap 메뉴 연결을 이어서 진행합니다

1. 하는 일

끌고 있는 위치(tick)를 **가장 가까운 격자 칸에 맞춥니다**. 격자 칸은 한 칸 길이(`stepTicks`)의 배수입니다.

> 맞춘 tick = (tick ÷ stepTicks 를 반올림한 값) × stepTicks

넣은 위치	한 칸	맞춘 결과
1,450	960	1,920 (가까운 오른쪽 칸)
400	960	0 (가까운 왼쪽 칸)
480	960	960 (정확히 반은 오른쪽)
3,840	960	3,840 (이미 격자 위)
130	80 (1/16 셋잇단)	160
-600	960	-960 (음수도 대칭)

2. 코드

```
export function snapTick(tick, stepTicks) {
  if (typeof tick !== 'number' || !isFinite(tick)) throw new RangeError('tick 은 유한한 숫자여야 합니다: ' + tick);
  if (typeof stepTicks !== 'number' || !isFinite(stepTicks) || stepTicks <= 0) throw new RangeError('stepTicks 는 0 보다 큰 숫자여야 합니다: ' + stepTicks);
  return Math.round(tick / stepTicks) * stepTicks;
}
```

최신 문법으로 쓰면 (공부용)

```
export const snapTick = (tick, stepTicks) => Math.round(tick / stepTicks) * stepTicks;
```

최신 문법	실제 코드	뜻
화살표 함수	`function` 선언	같은 동작입니다
`Number.isFinite`	`isFinite`	둘 다 되지만 `isFinite`는 숫자가 아닌 값도 검사합니다. 여기선 타입 검사를 앞에서 이미 합니다

3. 왜 이렇게

선택	이유
반올림(가장 가까운 칸)	FR-206 그리드 스냅의 기본 동작입니다. 왼쪽·오른쪽 중 가까운 쪽으로 붙습니다
`Math.round`	정확히 반(0.5)일 때 위로 올립니다. 격자가 두 칸 사이일 때 오른쪽으로 붙어 예측이 쉽습니다
`stepTicks <= 0` 오류	0으로 나누면 `Infinity`·`NaN`이 나 스냅이 망가집니다. 0 이하는 스냅 의미가 없어 바로 알립니다
tick·step 유한성 검사	끌기 도중 `NaN`이 들어오면 화면에서 클립이 사라집니다. 함수에서 막습니다
두 수를 곱해 정수 유지	`stepTicks`가 정수(240·960 등)라 결과도 정수입니다. 소수 오차(0.0000001)가 생기지 않습니다
음수도 그대로 반올림	클립을 마디 0 왼쪽으로 끌어도 값이 대칭입니다

4. 시험 (Node에서 직접 실행, 18개 통과)

확인	결과
위로(600→960)·아래로(400→0)·정확히 반(480→960)	✓
이미 맞은 점(960·3,840)은 그대로	✓
1/16 칸(240), 1/16 셋잇단 칸(80)으로 맞추기	✓
음수 대칭(-600→-960, -1,450→-1,920)	✓
step 0·음수, tick `NaN`, step 없음, tick 문자열 → 오류	✓

JS 121 — `moveClipStart()` (클립 이동 시작 위치)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 일곱 번째 함수 (FR-208 클립 이동)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	클립 길이 조절(오른쪽 끝 드래그)과 오디오 트리밍(왼쪽 끝 = `offset_tick`) 함수를 이어서 진행합니다

1. 하는 일

클립을 좌우로 끌었을 때 **새 시작 tick**을 돌려줍니다. 끈 거리를 더해 격자에 맞추고, **마디 0 왼쪽으로는 못 갑니다**.

> 새 시작 = max(0, 격자에 맞춘 (지금 시작 + 끈 거리))

지금 시작	끈 거리	한 칸	새 시작
960	0	960	960 (제자리)
960	+1,000	960	1,920
960	-700	960	0
0	-5,000	960	0 (0 아래로 안 내려감)
0	130	80 (1/16 셋잇단)	160
0	130	1 (Alt, 격자 무시)	130

2. 코드

```
export function moveClipStart(startTick, deltaTicks, stepTicks) {
  return Math.max(0, snapTick(startTick + deltaTicks, stepTicks));
}
```

최신 문법으로 쓰면 (공부용)

```
export const moveClipStart = (startTick, deltaTicks, stepTicks) =>
  Math.max(0, snapTick(startTick + deltaTicks, stepTicks));
```

최신 문법	실제 코드	뜻
화살표 함수	`function` 선언	같은 동작입니다

3. 왜 이렇게

선택	이유
`snapTick()`을 그대로 씀	격자 맞추기가 한 곳에서 나옵니다. `stepTicks`가 0이면 그 함수가 던진 오류가 전해집니다
`Math.max(0, …)`	마디 0 왼쪽은 시간이 없는 곳입니다. 클립이 사라지지 않게 여기서 막습니다
오른쪽 끝 제한은 안 함	여러 클립을 함께 옮기면 어디까지 갈 수 있는지가 달라집니다(프로토타입도 묶음 전체로 계산). 그래서 부르는 쪽에서 맡습니다
Alt(격자 무시)는 `stepTicks = 1`	격자를 무시할 땐 한 칸을 1틱으로 주면 됩니다. 별도 함수 없이 같은 코드로 됩니다
격자 벗어난 시작도 붙임	시작이 격자 밖이면(예: 100) 끌지 않아도 가까운 칸으로 붙습니다

4. 시험 (Node에서 직접 실행, 12개 통과)

확인	결과
제자리(960+0), 오른쪽(960+1,000→1,920), 왼쪽(960-700→0)	✓
0 아래로 못 감(0-5,000→0)	✓
오프그리드 시작(100)이 격자 0으로 붙음	✓
셋잇단 칸(80), Alt(step 1)·Alt 음수 → 0	✓
여러 칸 이동(0+4,000→3,840)	✓
step 0·delta `NaN` → 오류	✓

JS 122 — `resizeClipLength()` (클립 길이 조절)

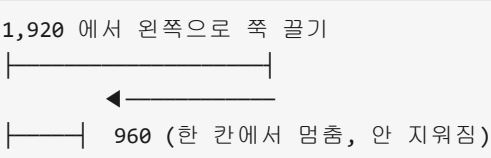
항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 여덟 번째 함수 (FR-208 길이 조절)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	오디오 클립 왼쪽 끝 트리밍(`offset_tick`) 함수와 복제(Ctrl+D)·분할 함수를 이어서 진행합니다

1. 하는 일

클립 **오른쪽 끝을 끌었을 때의 새 길이**를 돌려줍니다. 격자에 맞추고, **최소 길이는 한 칸**(Alt일 땐 1틱)을 지킵니다.

> 새 길이 = max(한 칸, 격자에 맞춘 (지금 길이 + 끈 거리))

지금 길이	끈 거리	한 칸	새 길이
960	0	960	960 (제자리)
960	+1,000	960	1,920
1,920	-700	960	960
1,920	-5,000	960	960 (한 칸에서 멈춤)
100	-500	1 (Alt)	1 (최소 1틱)
100	+37	1 (Alt)	137
240	-500	80 (1/16 셋잇단)	80



2. 코드

```
export function resizeClipLength(lengthTick, deltaTicks, stepTicks) {
  return Math.max(stepTicks, snapTick(lengthTick + deltaTicks, stepTicks));
}
```

최신 문법으로 쓰면 (공부용)

```
export const resizeClipLength = (lengthTick, deltaTicks, stepTicks) =>
  Math.max(stepTicks, snapTick(lengthTick + deltaTicks, stepTicks));
```

최신 문법	실제 코드	뜻
화살표 함수	`function` 선언	같은 동작입니다

3. 왜 이렇게

선택	이유
최소 = 한 칸(2026-10-14 사용자 결정)	스냅이 켜져 있으면 격자선까지만 줄어드는 것과 자연스럽게 맞습니다. 화면에서 계속 보입니다
Alt 는 최소 1틱	Alt 는 격자를 무시하니 한 칸이 1이라 그대로 1틱까지 줄어듭니다. 규칙을 따로 만들지 않아도 됩니다
`Math.max(stepTicks, …)`를 나중에	격자에 맞춘 뒤 올려서 결과가 항상 한 칸 이상이고, 격자 위 값입니다
클립을 지우지 않음	최소보다 더 끌어도 멈추기만 합니다. 삭제는 Delete 키가 맡아서 실수로 사라지지 않습니다
`snapTick()`을 그대로 씀	격자 맞추기가 한 곳에서 나오고, `stepTicks`가 0이면 그 오류가 전해집니다

4. 시험 (Node에서 직접 실행, 14개 통과)

확인	결과
제자리(0), 늘리기(960+1,000→1,920), 줄이기(1,920-700→960)	✓
한 칸 밑으로 줄여도 멈춤(1,920-5,000→960, 960-960→960)	✓
셋잇단 1/16 칸(80) 최소·늘리기	✓
Alt(step 1): 100-50→50, 최소 1틱, 늘리기 자유	✓
여러 마디 늘리기(960+3,900→4,800)	✓
step 0·delta `NaN` → 오류	✓

JS 123 — `trimClipLeft()` (왼쪽 끝 트리밍, 세 값 동시)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 아홉 번째 함수 (FR-208 오디오 트리밍)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	클립 복제(Ctrl+D)·분할 함수를 이어서 진행합니다

1. 하는 일

클립 **왼쪽 끝을 끌었을 때**의 새 값 세 개(`start_tick`·`length_tick`·`offset_tick`)를 한꺼번에 돌려줍니다. 오디오는 `offset_tick`(원본에서 건너뛰는 양)까지 같이 움직입니다.

- 줄이기: 오른쪽 끝과 같은 규칙. **최소 한 칸**, Alt면 1틱까지.
- 늘리기: **원본 처음**(`sourceStartTick`)과 **마디 0**에 닿을 때까지.

```
원본 오디오:  ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓
클립:                ┌──────────────┐
                     ↑ offset       ↑ offset+length

왼쪽으로 늘리기  ◀──┤   → 원본 처음(offset 0)이나 마디 0 에서 멈춤
왼쪽으로 줄이기     ├─▶   → 한 칸(length = stepTicks)에서 멈춤
```

start	length	offset	원본 시작	끈 거리	한 칸	결과 start·length·offset
960	960	960	0	-960	960	0 · 1,920 · 0
960	960	960	0	-3,840	960	0 · 1,920 · 0 (마디 0에서 멈춤)
960	960	480	480	-960	960	480 · 1,440 · 0 (원본 처음에서 멈춤)
960	1,920	960	0	+960	960	1,920 · 960 · 1,920
960	960	960	0	+5,000	960	960 · 960 · 960 (한 칸에서 멈춤)

MIDI처럼 원본이 없으면 `sourceStartTick`에 `null`을 넣습니다. 그러면 늘리기 한계는 마디 0뿐이고 `offset`은 늘 0입니다.

2. 코드

```
export function trimClipLeft(startTick, lengthTick, deltaTicks, stepTicks, sourceStartTick) {
  const minLength = stepTicks;
  const minStart = sourceStartTick == null ? 0 : Math.max(0, sourceStartTick);
  const maxStart = startTick + lengthTick - minLength;
  let start = snapTick(startTick + deltaTicks, stepTicks);
  if (start < minStart) start = minStart;
  if (start > maxStart) start = maxStart;
  return {
    start: start,
```

```
    length: lengthTick - (start - startTick),
    offset: sourceStartTick == null ? 0 : start - sourceStartTick,
  };
}
```

최신 문법으로 쓰면 (공부용)

```
export const trimClipLeft = (startTick, lengthTick, deltaTicks, stepTicks, sourceStartTick) => {
  const minLength = stepTicks;
  const minStart = sourceStartTick == null ? 0 : Math.max(0, sourceStartTick);
  const maxStart = startTick + lengthTick - minLength;
  const start = Math.min(Math.max(snapTick(startTick + deltaTicks, stepTicks), minStart), maxStart);
  return { start, length: lengthTick - (start - startTick), offset: sourceStartTick == null ? 0 : start
- sourceStartTick };
};
```

최신 문법	실제 코드	뜻
화살표 함수	`function` 선언	같은 동작입니다
중첩 삼항·`Math.min/max`를 한 줄에	`if` 두 줄	읽기 쉽게 풀어 쓴 것뿐입니다
`??` (null 병합)	`== null ?`	`null`과 `undefined`를 함께 거르되 `0`은 살립니다

3. 왜 이렇게

선택	이유
`offset`을 넣지 않고 결과로 돌려줌	오프셋은 `start - 원본시작`으로 저절로 정해집니다. 넣을 필요가 없습니다
`sourceStartTick` 하나로 오디오·MIDI 구분	원본이 없으면 `null`. 그러면 마디 0 한계만 남고 오프셋은 0이라 MIDI 동작이 됩니다
늘리기 = `minStart`로 clamp	원본 처음(`max(0, sourceStart)`)보다 왼쪽은 원본에 없는 부분이라 막습니다
줄이기 = `maxStart`로 clamp	`length = stepTicks`가 되는 지점입니다. 오른쪽 끝 함수와 최소 규칙이 같습니다
격자 맞춘 뒤 clamp	결과가 격자 근처에서 한계에 붙습니다. 한계값이 격자 밖이어도 정확히 한계에서 멈춥니다
세 값을 항상 함께 돌려줌	`start`가 움직인 만큼 `length`가 반대로, `offset`이 같이 움직여야 오디오가 제자리에 고정됩니다

4. 시험 (Node에서 직접 실행, 13개 통과)

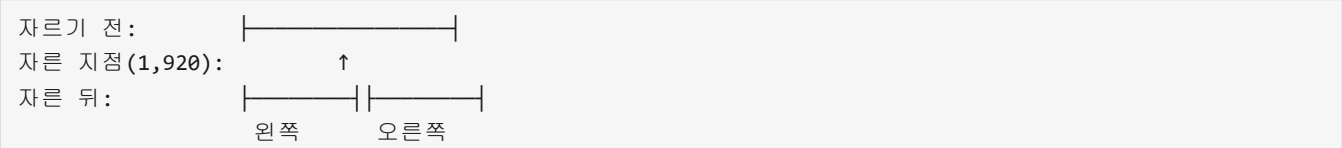
확인	결과
제자리 · 늘리기(960→0, 길이 1,920, offset 0)	✓
원본 처음 한계(sourceStart 480에서 멈춤) · 마디 0 한계	✓
왼쪽 줄이기(1,920−960→start 1,920·길이 960·offset 1,920)	✓
한 칸 밑으로 줄여도 멈춤 · Alt면 1틱까지	✓
MIDI(`sourceStartTick = null`): 마디 0까지 자유·offset 0	✓
step 0·delta `NaN` → 오류	✓

JS 124 — `splitClip()` (클립 분할)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 열 번째 함수 (FR-208 분할)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	클립 복제(Ctrl+D)는 `start + length` 한 줄이라 바로 이어서 넣습니다

1. 하는 일

클립을 **클릭 지점에서 둘로 가릅니다**. 자른 지점은 격자에 맞추고, 양쪽이 각각 **한 칸 이상**일 때만 가릅니다. 둘 중 하나라도 한 칸보다 짧으면 **가르지 않고 `null`**을 돌려줍니다.



자를 위치	한 칸	결과
1,920	960	왼쪽 {960, 960, 960} · 오른쪽 {1920, 960, 1920}
2,000	960	1,920로 붙어 위와 같음
960 (맨 앞)	960	`null` (왼쪽 0)
5,000 (끝 넘음)	960	`null` (오른쪽 0)
1,500	1 (Alt)	왼쪽 {960, 540, 960} · 오른쪽 {1500, 1380, 1500}
2,879	1 (Alt)	오른쪽이 1틱이라 분할됨

오디오는 자른 만큼 **오른쪽 토막의 오프셋이 커집니다**(원본에서 더 건너뜀). 오프셋 규칙(`sourceStartTick`)은 `trimClipLeft()`와 같습니다.

2. 코드

```
export function splitClip(startTick, lengthTick, atTick, stepTicks, sourceStartTick) {
  const at = snapTick(atTick, stepTicks);
  const leftLength = at - startTick;
  const rightLength = startTick + lengthTick - at;
  if (leftLength < stepTicks || rightLength < stepTicks) return null;
  const offset = (t) => (sourceStartTick == null ? 0 : t - sourceStartTick);
  return {
    left: { start: startTick, length: leftLength, offset: offset(startTick) },
    right: { start: at, length: rightLength, offset: offset(at) },
  };
}
```

최신 문법으로 쓰면 (공부용)

```
export const splitClip = (startTick, lengthTick, atTick, stepTicks, sourceStartTick) => {
  const at = snapTick(atTick, stepTicks);
  const leftLength = at - startTick;
  const rightLength = startTick + lengthTick - at;
  if (leftLength < stepTicks || rightLength < stepTicks) return null;
  const offset = (t) => (sourceStartTick == null ? 0 : t - sourceStartTick);
  return { left: { start: startTick, length: leftLength, offset: offset(startTick) }, right: { start: a
t, length: rightLength, offset: offset(at) } };
};
```

최신 문법	실제 코드	뜻
화살표 함수	`function` · 화살표	같은 동작입니다
`offset`을 화살표로 정의	그대로 사용	왼쪽·오른쪽 두 번 쓰려고 작은 함수로 묶었습니다

3. 왜 이렇게

선택	이유
자를 지점을 `snapTick()`으로 붙임	자른 자리가 격자 위에 있어야 두 토막이 다시 격자에 맞습니다
한 칸 미만이면 `null`	점처럼 작은 토막이 생기면 다시 잡기 어렵습니다. 못 가르면 아무 일도 안 일어난 게 낫습니다
`null`로 알림(오류 아님)	사용자가 클립 끝을 살짝 눌러 자르면 그냥 안 되는 게 자연스럽습니다. 빨간 오류가 뜨면 놀랍니다
오른쪽 오프셋 = `at - 원본시작`	왼쪽을 그대로 두고 오른쪽만 원본에서 더 건너뛰면 오디오가 제자리에 이어집니다
`sourceStartTick` 규칙 재사용	`trimClipLeft()`와 같은 규칙이라 MIDI(`null`)에서도 오프셋이 0으로 맞습니다
입력에 `offsetTick`이 없음	오프셋은 `start - 원본시작`으로 저절로 정해져 넣을 필요가 없습니다

4. 시험 (Node에서 직접 실행, 13개 통과)

확인	결과
가운데 자르기 · 격자로 붙어 자르기	✓
왼쪽·오른쪽 한 칸 미만 → `null` · 클립 밖 → `null`	✓
오디오 중간 시작(offset 480 → 오른쪽 1,440)	✓
MIDI(`null`) → 양쪽 offset 0	✓
Alt(step 1): 1,500·2,879 자르기, 끝(2,880) → `null`	✓
step 0·at `NaN` → 오류	✓

JS 125 — `duplicateClipStart()` (클립 복제 위치)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 열한 번째 함수 (FR-208 복제, Ctrl+D)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	트랙 추가·삭제(FR-207)와 가로·세로 줌(FR-209) 함수를 이어서 진행합니다

1. 하는 일

클립을 복제할 때 **복제본의 시작 tick**을 돌려줍니다. 복제본은 원본 **바로 뒤**에 붙습니다.

> 복제본 시작 = 원본 시작 + 원본 길이

```
원본:   ├──────────┼──────────┤   (시작 960 · 길이 1,920)
복제본:             ├──────────┼──────────┤   (시작 2,880)
```

원본 시작	원본 길이	복제본 시작
0	960	960
960	1,920	2,880
3,840	3,840	7,680
2,880	960	3,840

2. 코드

```
export function duplicateClipStart(startTick, lengthTick) {
  if (typeof startTick !== 'number' || !isFinite(startTick)) throw new RangeError('startTick 은 유한한 숫자여야 합니다: ' + startTick);
  if (typeof lengthTick !== 'number' || !isFinite(lengthTick) || lengthTick <= 0) throw new RangeError('lengthTick 은 0 보다 큰 숫자여야 합니다: ' + lengthTick);
  return startTick + lengthTick;
}
```

최신 문법으로 쓰면 (공부용)

```
export const duplicateClipStart = (startTick, lengthTick) => startTick + lengthTick;
```

최신 문법	실제 코드	뜻
화살표 함수	`function` 선언	같은 동작입니다

3. 왜 이렇게

선택	이유
원본 바로 뒤(`start + length`)	Ctrl+D의 익숙한 동작입니다. 빈 곳에 붙여 놓친 클립이 원본과 겹치지 않습니다
격자 스냅을 따로 안 함	시작·길이 둘 다 이미 격자 위 값이라 더한 값도 격자 위입니다. `snapTick()`을 부를 필요가 없습니다
길이 > 0 검사	길이가 0이면 복제본이 원본과 같은 자리에 겹칩니다. 0 이하는 잘못된 입력이라 막습니다
길이·오프셋은 안 돌려줌	원본과 같습니다. 시작만 알면 부르는 쪽이 같은 값으로 클립을 하나 더 만듭니다

4. 시험 (Node에서 직접 실행, 10개 통과)

확인	결과
이어 붙음(960+1,920→2,880, 3,840+3,840→7,680)	✓
앞에 붙음(0+960→960), 짧은 셋잇단 클립(0+320→320)	✓
격자 위 결과 유지(2,880+960→3,840)	✓
길이 0·음수, 시작 `NaN`·문자열, 길이 문자열 → 오류	✓

JS 126 — `clampZoom()` (가로 줌 한계)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 열두 번째 함수 (FR-209 가로 줌)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	커서 위치를 고정하는 줌(`zoomScrollLeft`)과 세로 줌(트랙 높이, FR-209) 함수를 이어서 진행합니다

1. 하는 일

가로 줌 **배율을 한계 안으로 맞춥니다.** 배율 1 = 1마디 28.8px이고, 클수록 마디가 넓어집니다(확대).

> 맞춘 배율 = 요청 배율을 [최소, 최대] 안으로 가둔 값 최소 = `ZOOM_MIN`(0.25) 과 **화면 맞춤 배율** 중 큰 쪽, 최대 = `ZOOM_MAX`(12)

요청	화면 맞춤	결과	이유
1	—	1	범위 안
0.1	—	0.25	최소보다 작음
20	—	12	최대보다 큼
0.5	0.8	0.8	화면 맞춤이 최소
5	0.8	5	둘 다보다 큼
0.1	0.1	0.25	화면 맞춤도 0.25 미만
1	100	12	화면 맞춤이 너무 큼(짧은 곡)

```
축소 ←── 0.25 ──┬──────────────────────┬── 12 ──→ 확대
                (화면 맞춤이 더 크면 그쪽)
```

2. 코드

```
export function clampZoom(zoom, fitZoom) {
  if (typeof zoom !== 'number' || !isFinite(zoom)) throw new RangeError('zoom 은 유한한 숫자여야 합니다:
' + zoom);
  const fit = (typeof fitZoom === 'number' && isFinite(fitZoom)) ? fitZoom : ZOOM_MIN;
  const lo = Math.min(ZOOM_MAX, Math.max(ZOOM_MIN, fit));
  return Math.min(Math.max(zoom, lo), ZOOM_MAX);
}
```

`ZOOM_MIN = 0.25`, `ZOOM_MAX = 12`는 파일 위쪽(`setZoom` 근처)에 이미 있는 값을 그대로 씁니다.

최신 문법으로 쓰면 (공부용)

```
export const clampZoom = (zoom, fitZoom = ZOOM_MIN) => {
  const lo = Math.min(ZOOM_MAX, Math.max(ZOOM_MIN, fitZoom));
  return Math.min(Math.max(zoom, lo), ZOOM_MAX);
};
```

최신 문법	실제 코드	뜻
기본값 인자 `fitZoom = ZOOM_MIN`	`typeof … ? fitZoom : ZOOM_MIN`	값을 안 줄 때의 기본값. 다만 `0`·`null` 같은 잘못된 값도 걸러야 해서 `typeof` 검사를 씁니다

3. 왜 이렇게

선택	이유
최소를 화면 맞춤으로	이보다 더 축소하면 곡 오른쪽에 빈 공간만 늘어납니다. 곡이 화면에 꼭 차는 게 가장 작은 씀씀이입니다
`fitZoom`을 [0.25, 12]로 눌러 씀	화면 맞춤이 12보다 큰 짧은 곡에서도 최소 ≤ 최대가 되어 결과가 한계 밖으로 새지 않습니다
`ZOOM_MIN`·`ZOOM_MAX` 재사용	프로토타입 `setZoom`과 같은 값입니다. 두 곳에 숫자를 두지 않습니다
`fitZoom` 없거나 잘못되면 `ZOOM_MIN`	화면 크기를 아직 못 잰 첫 렌더에서도 안전하게 0.25를 최소로 씁니다
배율 NaN·문자열은 오류	잘못된 배율로 화면을 그리면 격자가 사라집니다. 바로 알립니다

4. 시험 (Node에서 직접 실행, 14개 통과)

확인	결과
1은 그대로 · 0.1→0.25 · 20→12	✓
화면 맞춤이 최소(0.5, fit 0.8 → 0.8) · 그보다 크면 그대로	✓
fit 0.1→0.25 · fit 100→12 · fit 0→0.25 · fit `null`→0.25	✓
확대 요청도 12 상한	✓
zoom `NaN`·문자열 → 오류	✓

JS 127 — `zoomScrollLeft()` (마우스 지점 고정 줌)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 열세 번째 함수 (FR-209 가로 줌)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	세로 줌(트랙 높이, FR-209)과 트랙 추가·삭제(FR-207) 함수를 이어서 진행합니다

1. 하는 일

가로 줌으로 **마디 폭이 바뀔 때, 마우스 아래 지점이 화면에서 안 움직이게** 할 스크롤 위치를 돌려줍니다.

> 1. 마우스 아래의 마디 위치를 구합니다: `(스크롤 + 마우스x − 헤드폭) ÷ 옛 마디폭`
> 2. 폭이 바뀐 뒤 같은 화면 x 에 오도록 스크롤을 옮깁니다: `헤드폭 + 마디위치 × 새 마디폭 − 마우스x`

```
확대 전:  |──100──|──100──|     마우스가 마디 1 위
스크롤 0,  마우스 x=300,  헤드 200
확대 후:  |──200──|──200──|     마디 1 이 안 움직이게 스크롤 100
```

스크롤	마우스x	헤드	옛 폭	새 폭	새 스크롤
0	300	200	100	200	100 (확대)
50	200	200	100	200	100 (헤드 끝 지점 고정)
400	500	200	200	100	50 (축소)
123	456	200	150	150	123 (폭 그대로)

2. 코드

```
export function zoomScrollLeft(scrollLeft, anchorX, headWidth, oldBarWidth, newBarWidth) {
  const nums = [scrollLeft, anchorX, headWidth, newBarWidth];
  if (nums.some((n) => typeof n !== 'number' || !isFinite(n))) throw new RangeError('줌 스크롤 인자는 유한한 숫자여야 합니다');
  if (typeof oldBarWidth !== 'number' || !isFinite(oldBarWidth) || oldBarWidth <= 0) throw new RangeError('oldBarWidth 는 0 보다 큰 숫자여야 합니다: ' + oldBarWidth);
  const barAtAnchor = (scrollLeft + anchorX - headWidth) / oldBarWidth;
  return headWidth + barAtAnchor * newBarWidth - anchorX;
}
```

최신 문법으로 쓰면 (공부용)

```
export const zoomScrollLeft = (scrollLeft, anchorX, headWidth, oldBarWidth, newBarWidth) => {
  const barAtAnchor = (scrollLeft + anchorX - headWidth) / oldBarWidth;
  return headWidth + barAtAnchor * newBarWidth - anchorX;
};
```

최신 문법	실제 코드	뜻
화살표 함수	`function` 선언	같은 동작입니다
인자 검사를 안 씀	`typeof`·`isFinite` 검사	잘못된 값(0 나눗셈 등)을 막으려고 검사를 넣었습니다

3. 왜 이렇게

선택	이유
헤드 폭을 빼고 계산	왼쪽 트랙 이름칸은 줌과 무관하게 고정입니다. 마디가 시작하는 곳부터 재야 마우스 지점이 정확합니다
`oldBarWidth > 0` 검사	0으로 나누면 `Infinity`가 나 스크롤이 망가집니다
결과를 안 자름(음수도 그대로)	이 함수는 좌표 계산만 합니다. 실제로 보이는 범위로 자르는 일은 스크롤 영역(`scrollLeft`)이 합니다
`nums.some`으로 네 값 검사	`isFinite('200')`은 문자열을 숫자로 바꿔 통과시킵니다. `typeof`까지 봐야 잘못된 값이 걸립니다
폭이 같으면 스크롤 그대로	확대·축소가 아니면 화면이 안 움직여야 합니다. 식이 저절로 그렇게 나옵니다

4. 시험 (Node에서 직접 실행, 11개 통과)

확인	결과
확대(0→100) · 헤드 끝 지점 고정(→100) · 축소(→50)	✓
폭 그대로면 스크롤 그대로(123)	✓
헤드 0, 음수 결과도 그대로	✓
`oldBarWidth` 0·음수, `anchorX` `NaN`, 새 폭 문자열 → 오류	✓

JS 128 — `stepTrackHeight()` (세로 줌 트랙 높이 단계)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 열네 번째 함수 (FR-209 세로 줌)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	트랙 추가·삭제(FR-207) 함수를 이어서 진행합니다

1. 하는 일

세로 줌: 트랙 높이를 **한 단계 키우거나 줄인 다음 높이**를 돌려줍니다. 규칙은 프로토타입에 이미 있던 것을 그대로 씁니다.

```
트랙 높이 단계 (px)
 28 ── 36 ── 44 ── 56 ── 72 ── 96 ── 128 ── 160 ── 200
최소       (기본)                                  최대
            └ 38 미만이면 트랙 이름 줄 숨김(compact)
```

지금 높이	방향	다음
44	크게	56
44	작게	36
50	크게	56
50	작게	44
28	작게	`null` (최소)
200	크게	`null` (최대)

2. 코드

```
export function stepTrackHeight(currentPx, dir) {
  if (typeof currentPx !== 'number' || !isFinite(currentPx)) throw new RangeError('currentPx 는 유한한 숫자여야 합니다: ' + currentPx);
  const up = dir > 0;
  const next = up
    ? TRACK_H_STEPS.find((h) => h > currentPx + 0.5)
    : TRACK_H_STEPS.slice().reverse().find((h) => h < currentPx - 0.5);
  return next == null ? null : next;
}
```

`TRACK_H_STEPS = [28, 36, 44, 56, 72, 96, 128, 160, 200]`은 파일 아래쪽(트랙 높이 기능)에 이미 있는 값을 그대로 씁니다.

최신 문법으로 쓰면 (공부용)

```
export const stepTrackHeight = (currentPx, dir) =>
  (dir > 0 ? TRACK_H_STEPS.find((h) => h > currentPx + 0.5)
           : [...TRACK_H_STEPS].reverse().find((h) => h < currentPx - 0.5)) ?? null;
```

최신 문법	실제 코드	뜻
`?? null` (null 병합)	`next == null ? null : next`	`find`가 못 찾으면 `undefined`를 돌려주는데, 밖으로는 `null`로 맞춥니다
스프레드 `[...arr]`	`arr.slice()`	배열을 복사해 뒤집습니다(원본을 안 건드림)

3. 왜 이렇게

선택	이유
단계 목록을 그대로 씀	프로토타입에서 이미 정한 값(28~200, 44 기본)입니다. 화면 CSS `--sq-row-h`와 같은 값이라 눈금이 맞습니다
0.5px 여유	지금 높이가 단계 사이(예: 50.3)여도 반올림 오차로 단계를 건너뛰지 않게 합니다
못 가면 `null`(오류 아님)	이미 최소·최대에 닿은 것은 정상입니다. 부르는 쪽은 `null`이면 아무것도 안 하면 됩니다
방향은 `dir > 0`만 봄	0이나 음수는 줄이기로 봅니다. `null` 없이도 동작합니다
`currentPx` 검사	화면을 못 잰 첫 순간에는 높이가 `NaN`일 수 있어 막습니다

4. 시험 (Node에서 직접 실행, 16개 통과)

확인	결과
44→56, 44→36, 28→36, 200→160	✓
28 -1·200 +1 → `null`	✓
단계 사이(50·45·44.4)에서 위·아래 이동	✓
방향 0·음수는 줄이기	✓
연속 세 번 크게(44→56→72→96)	✓
`currentPx` `NaN`·문자열 → 오류	✓

JS 129 — `nextTrackNumber()` (새 트랙 번호)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 열다섯 번째 함수 (FR-207 트랙 추가)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	트랙 삭제 뒤 번호를 다시 매기는 `renumberMap()` 함수를 이어서 진행합니다

1. 하는 일

트랙을 새로 추가할 때 **붙일 번호**를 돌려줍니다. 번호 = 지금 트랙 번호 중 **가장 큰 값 + 1**입니다.

> 새 번호 = (지금까지 가장 큰 번호) + 1, 트랙이 없으면 1

지금 번호	새 번호	이유
`[]`	1	트랙 없음
`[1, 2, 3]`	4	가장 큰 값 3 + 1
`[1, 3]`	4	2가 비어도 가장 큰 값 기준
`[3, 1, 2]`	4	순서와 무관
`[100, 3]`	101	가장 큰 값 100 + 1
`[1, null, 2]`	3	그룹 트랙(번호 없음)은 건너뜀

```
1. Kick   2. Bass   3. Lead   →  [+ Track]  4. ...
```

2. 코드

```
export function nextTrackNumber(numbers) {
  if (!Array.isArray(numbers)) throw new RangeError('numbers 는 배열이어야 합니다');
  let max = 0;
  for (const n of numbers) {
    if (typeof n === 'number' && isFinite(n) && n > max) max = n;
  }
  return max + 1;
}
```

최신 문법으로 쓰면 (공부용)

```
export const nextTrackNumber = (numbers) => Math.max(0, ...numbers.filter(Number.isFinite)) + 1;
```

최신 문법	실제 코드	뜻
`Math.max(0, ...arr)` 스프레드	`for` 반복	같은 결과입니다. 목록이 아주 길 때 스프레드가 인자 한계에 걸릴 수 있어 반복으로 씁니다
`Number.isFinite`	`isFinite`	둘 다 되지만 여기선 `typeof`까지 함께 봅니다

3. 왜 이렇게

선택	이유
가장 큰 값 + 1	프로토타입 `createTrack()`이 쓰던 규칙 그대로입니다. 중간 번호가 비어도(삭제 전 상태) 겹치지 않게 붙습니다
번호 없는 줄은 셈하지 않음	그룹 트랙처럼 번호가 없는 줄이 끼어도 번호가 밀리지 않습니다
`for` 반복	`Math.max(0, …numbers)`보다 안전하고, 숫자가 아닌 값도 자연스럽게 걸러집니다
배열이 아니면 오류	트랙 목록을 잘못 넘기면 조용히 1을 돌려줘 번호가 겹칩니다. 바로 알립니다

4. 시험 (Node에서 직접 실행, 12개 통과)

확인	결과
없음 → 1 · [1,2,3] → 4 · [1,3] → 4 · [7] → 8	✓
중복([1,2,2]→3) · 순서 뒤섞임([3,1,2]→4)	✓
그룹 줄 건너뜀([1,null,2]→3) · 0은 안 셈([0]→1)	✓
문자열 무시(["5",1]→2) · 큰 번호([100,3]→101)	✓
배열 아님·없음 → 오류	✓

JS 130 — `renumberMap()` (트랙 삭제 뒤 번호 정리표)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 열여섯 번째 함수 (FR-207 트랙 삭제)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	트랙 순서 드래그(FR-207)와 오디오 클립 파형(FR-210) 함수를 이어서 진행합니다

1. 하는 일

트랙을 지운 뒤 번호를 다시 매긴 **표(옛 번호 → 새 번호)**를 돌려줍니다. 남은 트랙을 위에서부터 차례로 1, 2, 3 … 으로 매깁니다(번호 = 위치).

```
2번(Bass) 삭제
전:  1. Kick   2. Bass   3. Lead   4. Pad
후:  1. Kick             2. Lead   3. Pad
     표: 1→1,  3→2,  4→3
```

지금 번호	새 번호 표
`[1, 2, 3]`	1→1, 2→2, 3→3
`[1, 3, 4]`	1→1, 3→2, 4→3
`[3, 4, 5]`	3→1, 4→2, 5→3
`[1, null, 3]`	1→1, 3→2 (그룹 줄 건너뜀)
`[2]`	2→1
`[]`	(빈 표)

2. 코드

```
export function renumberMap(numbers) {
  if (!Array.isArray(numbers)) throw new RangeError('numbers 는 배열이어야 합니다');
  const map = new Map();
  let n = 0;
  for (const old of numbers) {
    if (typeof old !== 'number' || !isFinite(old) || old <= 0) continue;
    n += 1;
    map.set(old, n);
  }
  return map;
}
```

최신 문법으로 쓰면 (공부용)

```
export const renumberMap = (numbers) =>
```

```
  new Map(numbers.filter((x) => Number.isFinite(x) && x > 0).map((x, i) => [x, i + 1]));
```

최신 문법	실제 코드	뜻
`filter` + `map`	`for` 반복 + `continue`	같은 결과입니다. `null`·문자열을 거르는 조건이 한눈에 보이도록 반복으로 씁니다
`Number.isFinite`	`isFinite`	둘 다 되지만 여기선 `typeof`까지 함께 봅니다

3. 왜 이렇게

선택	이유
번호 = 위치(1..n)	프로토타입 규칙 그대로입니다. Ableton·FL처럼 중간 번호를 다시 채워 빈 번호가 남지 않습니다
그룹 줄 건너뜀	번호가 없는 줄(그룹 트랙)은 세지 않습니다. 번호 있는 트랙만 차례로 채웁니다
표(Map)로 돌려줌	번호를 가리키는 곳(↤10 · Track 10 등)을 한 번에 바꾸려면 옛→새 표가 필요합니다. 프로토타입 `renumberTracks()`와 같은 뜻입니다
`Map`은 넣은 차례를 지킴	위에서 아래 순서가 그대로 남아 새 번호가 순증합니다
배열이 아니면 오류	목록을 잘못 넘기면 조용히 빈 표가 나와 번호가 안 바뀝니다. 바로 알립니다

4. 시험 (Node에서 직접 실행, 11개 통과)

확인	결과
차례대로는 그대로(1→1,2→2,3→3)	✓
중간 삭제(1→1,3→2,4→3) · 뒷 번호(3→1,4→2,5→3)	✓
그룹 줄 건너뜀(한 줄·여러 줄)	✓
하나만(2→1) · 빈 목록(빈 표)	✓
0·음수·문자열은 건너뜀	✓
배열 아님·없음 → 오류	✓

JS 131 — `reorderItems()` (트랙 순서 바꾸기)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 열일곱 번째 함수 (FR-207 트랙 순서 드래그)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	트랙 이름·색상(FR-207)과 오디오 클립 파형(FR-210) 함수를 이어서 진행합니다

1. 하는 일

목록에서 항목 하나를 **다른 자리로 옮긴 새 목록**을 돌려줍니다. 트랙을 위아래로 끌어 순서를 바꿀 때 씁니다.

> `from` 자리 항목을 빼서 `to` 자리에 끼워 넣습니다. `to`는 **옮긴 뒤 자리**입니다.

```
[Kick, Bass, Lead] 에서 0번을 2번으로
   Kick 빼기 → [Bass, Lead]
   2번에 끼우기 → [Bass, Lead, Kick]
```

원래	from	to	결과
[Kick, Bass, Lead]	0	2	[Bass, Lead, Kick]
[Kick, Bass, Lead]	2	0	[Lead, Kick, Bass]
[Kick, Bass, Lead]	1	1	[Kick, Bass, Lead]
[1, 2, 3, 4]	0	3	[2, 3, 4, 1]

번호는 자리에서 나오므로, 옮긴 뒤 `renumberMap()`으로 다시 매깁니다.

2. 코드

```
export function reorderItems(items, from, to) {
  if (!Array.isArray(items)) throw new RangeError('items 는 배열이어야 합니다');
  const n = items.length;
  if (!Number.isInteger(from) || from < 0 || from >= n) throw new RangeError('from 자리가 범위 밖입니다:
' + from);
  if (!Number.isInteger(to) || to < 0 || to >= n) throw new RangeError('to 자리가 범위 밖입니다: ' + to);
  const out = items.slice();
  const moved = out.splice(from, 1)[0];
  out.splice(to, 0, moved);
  return out;
}
```

최신 문법으로 쓰면 (공부용)

```
export const reorderItems = (items, from, to) => {
  const out = [...items];
  out.splice(to, 0, ...out.splice(from, 1));
  return out;
};
```

최신 문법	실제 코드	뜻
스프레드 복사 `[...items]`	`items.slice()`	같은 뜻입니다
`...out.splice(...)` 한 줄	`moved` 변수 두 줄	같은 동작입니다. 읽기 쉽게 나눴습니다
`Array.prototype.toSpliced`(최신)	`slice` + `splice`	원본을 안 바꾸는 최신 메서드입니다. 여기선 옛 방식으로 씁니다

3. 왜 이렇게

선택	이유
`to` = 옮긴 뒤 자리	"빼고 나서 끼운 자리"로 정하면 같은 자리(1→1)가 저절로 제자리가 됩니다. 머릿속으로 따라가기 쉽습니다
원본을 안 바꿈(`slice`)	끌기 도중 실패해도 원래 순서가 남습니다. 되돌리기가 쉽습니다
`from`·`to` 정수 검사	소수 자리나 범위 밖이 들어오면 `splice`가 엉뚱하게 잘라 순서가 망가집니다. 바로 막습니다
자리 범위를 `[0, n)`으로	자리는 항목 사이가 아니라 항목이 있는 자리입니다. 하나만 있을 때 0→0만 허용됩니다
번호와 분리	순서 바꾸기와 번호 매기기(`renumberMap`)를 따로 두면 각각 시험하기 쉽습니다

4. 시험 (Node에서 직접 실행, 15개 통과)

확인	결과
앞으로(0→2)·뒤로(2→0)·제자리(1→1)	✓
한 칸 이동(1→2, 2→1) · 가운데(1→2) · 맨 끝(0→3)	✓
원래 목록이 안 바뀜	✓
하나짜리(0→0)	✓
범위 밖·음수·소수·배열 아님·빈 목록 → 오류	✓

JS 132 — `trackColor()` (트랙 색 고르기)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 열여덟 번째 함수 (FR-207 트랙 색상)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	오디오 클립 파형(FR-210) 함수를 이어서 진행합니다

1. 하는 일

트랙에 붙일 **색 이름**을 돌려줍니다.

- 오디오 트랙은 늘 `'audio'` (파형 색 한 가지)
- MIDI 트랙은 고른 색이 있으면 그것, 없으면 **번호로 다섯 색을 돌려 씁니다**

> MIDI 기본 색 = `((번호 - 1) % 5) + 1` → 1·2·3·4·5 뒤에 1·2·3·4·5 …

종류	번호	고른 색	결과
audio	아무거나	—	`'audio'`
midi	1	—	`'1'`
midi	5	—	`'5'`
midi	6	—	`'1'` (돌아옴)
midi	2	`'3'`	`'3'`
midi	3	`''`	`'3'` (빈 값은 무시)

```
1  2  3  4  5  6  7  8  9  10
1  2  3  4  5  1  2  3  4  5   ← MIDI 색 번호
```

2. 코드

```
export function trackColor(kind, num, color) {
  if (kind === 'audio') return 'audio';
  if (typeof num !== 'number' || !isFinite(num) || num < 1) throw new RangeError('트랙 번호는 1 이상이어야 합니다: ' + num);
  return color || String(((num - 1) % 5) + 1);
}
```

최신 문법으로 쓰면 (공부용)

```
export const trackColor = (kind, num, color) =>
  kind === 'audio' ? 'audio' : (color || String(((num - 1) % 5) + 1));
```

최신 문법	실제 코드	뜻
삼항 연산자 한 줄	`if` + `return`	같은 동작입니다. 번호 검사를 따로 두려고 풀어 썼습니다

3. 왜 이렇게

선택	이유
오디오는 `'audio'`	파형은 색 한 가지로 보여 줍니다. 프로토타입 규칙 그대로입니다
`((번호 - 1) % 5) + 1`	다섯 색을 차례로 돌려 씁니다. 번호 1이 첫 색이 되도록 1을 뺍니다
고른 색 우선	사용자가 고른 색이 있으면 번호 색을 덮습니다. `color
색 이름은 숫자 문자열	실제 색값은 CSS(`--tc-1`~`--tc-5`)가 정합니다. JS는 어느 색인지만 알려 줍니다
MIDI 번호 1 미만은 오류	번호 0이면 `% 5` 가 음수가 나와 `'0'` 같은 없는 색이 됩니다. 바로 막습니다

4. 시험 (Node에서 직접 실행, 15개 통과)

확인	결과
오디오는 번호 무관하게 `'audio'`	✓
MIDI 1~5 → 1~5	✓
MIDI 6→1·7→2·10→5·11→1 (다섯 색 순환)	✓
고른 색 우선(번호 무관) · 빈 문자열은 무시	✓
MIDI 번호 0·`NaN`·없음 → 오류	✓

JS 133 — `waveformColumns()` (오디오 클립 파형 접기)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 열아홉 번째 함수 (FR-210 오디오 클립 파형)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	파형을 캔버스에 그리는 함수(그리기)를 이어서 진행합니다

1. 하는 일

오디오 클립의 파형을 **클립 폭(칸)에 맞게 접어** 한 칸마다 진폭을 돌려줍니다.

- `peaks`(원본 소스 전체 진폭)를 `sourceLengthTick`에 걸쳐 있다고 봅니다.
- 클립은 소스의 `offsetTick`부터 `lengthTick`만큼만 보여 줍니다(트리밍 반영).
- 그 구간을 가로 `columns`칸으로 나누고, **한 칸이 덮는 구간의 가장 큰 진폭**(절댓값)을 그 칸 값으로 씁니다.

```
peaks  [0.1, 0.5, 0.9, 0.3]   소스 4틱, offset 0, 길이 4
4칸  → [0.1, 0.5, 0.9, 0.3]     (그대로)
2칸  → [0.5, 0.9]               (큰 값만)
8칸  → [0.1,0.1, 0.5,0.5, 0.9,0.9, 0.3,0.3]
```

peaks	offset	길이	소스	칸	결과
[0.1,0.5,0.9,0.3]	0	4	4	2	[0.5, 0.9]
[0.1,0.5,0.9,0.3]	1	2	4	2	[0.5, 0.9]
[0.1,0.5,0.9,0.3]	2	2	4	1	[0.9]
[-0.7, 0.2]	0	2	2	2	[0.7, 0.2]
[]	0	4	4	3	[0, 0, 0]

2. 코드

```
export function waveformColumns(peaks, offsetTick, lengthTick, sourceLengthTick, columns) {
  if (!Array.isArray(peaks)) throw new RangeError('peaks 는 배열이어야 합니다');
  if (!Number.isInteger(columns) || columns < 1) throw new RangeError('columns 는 1 이상의 정수여야 합니다: ' + columns);
  if (typeof sourceLengthTick !== 'number' || !isFinite(sourceLengthTick) || sourceLengthTick <= 0) throw new RangeError('sourceLengthTick 는 0 보다 큰 숫자여야 합니다: ' + sourceLengthTick);
  const out = new Array(columns).fill(0);
  const n = peaks.length;
  if (n === 0) return out;
  const peaksPerTick = n / sourceLengthTick;
  for (let c = 0; c < columns; c++) {
    const t0 = (offsetTick + (c / columns) * lengthTick) * peaksPerTick;
    const t1 = (offsetTick + ((c + 1) / columns) * lengthTick) * peaksPerTick;
    let i0 = Math.floor(t0);
    let i1 = Math.ceil(t1);
```

```
    if (i1 <= i0) i1 = i0 + 1;
    i0 = Math.max(0, Math.min(n - 1, i0));
    i1 = Math.max(i0 + 1, Math.min(n, i1));
    let max = 0;
    for (let i = i0; i < i1; i++) {
      const v = Math.abs(peaks[i]);
      if (v > max) max = v;
    }
    out[c] = max;
  }
  return out;
}
```

최신 문법으로 쓰면 (공부용)

```
export function waveformColumns(peaks, offsetTick, lengthTick, sourceLengthTick, columns) {
  const inRange = peaks.slice(); // 그대로
  return Array.from({ length: columns }, (_, c) => {
    const at = (k) => (offsetTick + (k / columns) * lengthTick) * (peaks.length / sourceLengthTick);
    const i0 = Math.max(0, Math.floor(at(c)));
    const i1 = Math.max(i0 + 1, Math.min(peaks.length, Math.ceil(at(c + 1))));
    return Math.max(0, ...peaks.slice(i0, i1).map(Math.abs));
  });
}
```

최신 문법	실제 코드	뜻
`Array.from({length}, ...)`	`for` + `out[c]`	같은 결과입니다. 칸마다 계산을 눈으로 따라가기 쉽게 반복으로 씁니다
`Math.max(0, ...배열)`	`for` + 비교	큰 배열에서는 스프레드가 인자 한계에 걸립니다. 반복이 안전합니다
화살표 함수 `at`	식을 그대로 폄	같은 식입니다

3. 왜 이렇게

선택	이유
소스 좌표 → 칸 좌표 두 번 나눔	`tick → peak` 인덱스로 한 번, `칸 → tick`으로 한 번. 트리밍(offset)이 있든 없든 같은 식으로 처리됩니다
한 칸 = 구간의 최댓값	파형을 줄일 때 큰 소리가 사라지면 안 됩니다. 대표값으로 최댓값을 씁니다
절댓값(`Math.abs`)	파형은 위아래 어느 쪽이든 크기가 중요합니다. 부호는 그릴 때 위/아래로 다시 나눕니다
칸보다 peak가 적으면 이웃으로 채움	`i1 <= i0`이면 한 칸이 peak 하나보다 좁습니다. 가장 가까운 peak 하나를 씁니다
빈 peaks는 0으로	파형이 없어도 클립 폭만큼 자리를 잡아 둡니다(레이아웃 흔들림 방지)

4. 시험 (Node에서 직접 실행, 13개 통과)

확인	결과
칸수=peak수(그대로) · 절반(큰 값) · 두 배(각 peak 두 칸)	✓
offset·길이로 일부만(2칸·1칸) · 소스가 클립보다 길 때	✓
음수는 절댓값으로	✓

확인	결과
빈 peaks → 0으로 채움	✓
결과 길이 = columns	✓
peaks 배열 아님·칸 0·칸 소수·소스길이 0 → 오류	✓

JS 134 — `waveformBars()` (파형 위·아래 좌표)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 스무 번째 함수 (FR-210 오디오 클립 파형 그리기)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	파형을 실제 캔버스에 그리는 배선(그리기 함수)과 클립 배치를 진행합니다

1. 하는 일

`waveformColumns()`가 만든 칸 진폭(0~1)을 **가운데를 기준으로 위·아래 좌표**로 펼칩니다. 캔버스에 세로 선으로 그릴 때 바로 쓰는 좌표입니다.

- 가운데 = `height / 2`
- 각 칸: `top = 가운데 − 진폭 × 반`, `bottom = 가운데 + 진폭 × 반`
- 캔버스 그리기: `x = 칸 번호 × 칸 너비` 로 두고 `moveTo(x, top)` → `lineTo(x, bottom)`

```
칸 [1, 0, 0.5], 높이 10 (가운데 5)
  1.0 → [0, 10]      ■
  0.0 → [5,  5]      −
  0.5 → [2.5, 7.5]   ■
```

칸	높이	결과
[1, 0, 0.5]	10	[[0, 10], [5, 5], [2.5, 7.5]]
[1, 0.25]	8	[[0, 8], [3, 5]]
[2] (1로 잘림)	10	[[0, 10]]
[-1] (0으로)	10	[[5, 5]]
[]	10	[]

2. 코드

```
export function waveformBars(columns, height) {
  if (!Array.isArray(columns)) throw new RangeError('columns 는 배열이어야 합니다');
  if (typeof height !== 'number' || !isFinite(height) || height <= 0) throw new RangeError('height 는 0
보다 큰 숫자여야 합니다: ' + height);
  const midY = height / 2;
  const out = new Array(columns.length);
  for (let i = 0; i < columns.length; i++) {
    let amp = columns[i];
    if (typeof amp !== 'number' || !isFinite(amp) || amp < 0) amp = 0;
    if (amp > 1) amp = 1;
    out[i] = [midY - amp * midY, midY + amp * midY];
  }
```

```
  return out;
}
```

최신 문법으로 쓰면 (공부용)

```
export const waveformBars = (columns, height) =>
  columns.map((raw) => {
    const amp = Math.min(1, Math.max(0, Number.isFinite(raw) ? raw : 0));
    return [height / 2 - amp * height / 2, height / 2 + amp * height / 2];
  });
```

최신 문법	실제 코드	뜻
`map`	`for` 반복	같은 결과입니다. 진폭 자르는 단계가 눈에 보이도록 반복으로 씁니다
`Number.isFinite`	`typeof` + `isFinite`	숫자 아님까지 거르려고 둘 다 봅니다

3. 왜 이렇게

선택	이유
가운데 기준 대칭	파형은 위아래로 뻗습니다. 가운데 줄을 기준으로 삼으면 진폭 0이 곧 무음(가운데 선)이 됩니다
`[top, bottom]` 짝으로 돌려줌	캔버스 `moveTo`·`lineTo`에 그대로 넣습니다. 칸 너비(가로)는 부르는 쪽이 곱합니다
진폭 0~1로 자름	샘플 진폭이 넘칠 수 있습니다. 화면을 벗어나지 않게 클램프합니다
소리 없는 칸은 한 줄	완전히 지우지 않고 가운데 선을 남겨, 클립이 빈 곳 없이 이어져 보입니다
높이만 받음(가로 없음)	가로 위치는 칸 번호로 정해집니다. 이 함수는 세로 모양만 맡습니다

4. 시험 (Node에서 직접 실행, 14개 통과)

확인	결과
칸 [1,0,0.5]·높이 10 → [[0,10],[5,5],[2.5,7.5]]	✓
높이 8(가운데 4) · 최대 진폭은 위아래 끝 · 0은 가운데 줄	✓
진폭 >1은 1로 · 음수는 0으로 · 숫자 아님은 0으로	✓
빈 칸 → 빈 배열 · 결과 길이 = 칸 수 · `top ≤ bottom`	✓
칸 배열 아님·높이 0·음수·숫자 아님 → 오류	✓

JS 135 — `drawWaveform()` (파형 캔버스에 그리기)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 스물한 번째 함수 (FR-210 오디오 클립 파형 그리기)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	캔버스 크기 계산과 클립 배치를 잇는 배선을 진행합니다

1. 하는 일

`waveformBars()`가 만든 `[top, bottom]` 좌표를 **캔버스에 세로 선으로 그립니다**. 칸마다 `moveTo` → `lineTo`로 막대 하나를 긋습니다.

- 칸의 x = **칸 가운데** (`칸 번호 × 칸 너비 + 칸 너비 / 2`)
- 색·굵기(`strokeStyle`·`lineWidth`)는 부르는 쪽이 미리 정합니다

```
bars [[0,10],[5,5]], 칸 너비 4
  x=2 에 [0 → 10]
  x=6 에 [5 → 5]
```

호출 순서: `beginPath` → (칸마다 `moveTo`·`lineTo`) → `stroke`

2. 코드

```
export function drawWaveform(ctx, bars, colWidth) {
  if (!ctx || typeof ctx.beginPath !== 'function') throw new RangeError('ctx 는 캔버스 2D 컨텍스트여야 합니다');
  if (!Array.isArray(bars)) throw new RangeError('bars 는 배열이어야 합니다');
  if (typeof colWidth !== 'number' || !isFinite(colWidth) || colWidth <= 0) throw new RangeError('colWidth 는 0 보다 큰 숫자여야 합니다: ' + colWidth);
  ctx.beginPath();
  for (let i = 0; i < bars.length; i++) {
    const x = i * colWidth + colWidth / 2;
    const bar = bars[i];
    ctx.moveTo(x, bar[0]);
    ctx.lineTo(x, bar[1]);
  }
  ctx.stroke();
}
```

최신 문법으로 쓰면 (공부용)

```
export const drawWaveform = (ctx, bars, colWidth) => {
  ctx.beginPath();
  bars.forEach(([top, bottom], i) => {
    const x = i * colWidth + colWidth / 2;
```

```
    ctx.moveTo(x, top);
    ctx.lineTo(x, bottom);
  });
  ctx.stroke();
};
```

최신 문법	실제 코드	뜻
`forEach` + 구조 분해 `([top, bottom])`	`for` + `bar[0]`·`bar[1]`	같은 결과입니다. 좌표가 어디서 오는지 반복문으로 또렷하게 읽히게 씁니다

3. 왜 이렇게

선택	이유
`beginPath`·`stroke`를 함수 안에서	부르는 쪽이 잊어도 선이 반드시 그려집니다. 그리기 단위가 스스로 완결됩니다
x = 칸 가운데	막대가 칸 한가운데 서야 칸 너비가 커져도 고르게 보입니다. 한쪽 끝에 붙지 않습니다
색·굵기는 안 정함	이 함수는 **모양**만 맡습니다. 트랙 색(FR-207 `trackColor`)·선 굵기는 부르는 쪽이 정합니다
`ctx.beginPath` 존재 확인	`ctx`가 없거나 캔버스가 아니면 중간에 조용히 멈춥니다. 시작 전에 막습니다
빈 배열이면 선 없이 스트로크	파형이 없어도 정상으로 끝나, 뒤 이어지는 그리기에 지장이 없습니다

4. 시험 (Node에서 직접 실행, 12개 통과)

가짜 캔버스(호출을 기록하는 객체)를 만들어 확인했습니다.

확인	결과
그리기 순서(beginPath→moveTo→lineTo→…→stroke)	✓
`moveTo`·`lineTo` 수 = 막대 수 · `beginPath`·`stroke` 각 한 번	✓
빈 막대 → `beginPath`·`stroke`만	✓
칸 너비 1 → x=0.5 (칸 가운데)	✓
ctx 없음·`beginPath` 없음·bars 배열 아님·칸 너비 0·숫자 아님 → 오류	✓

JS 136 — `snapOptions()` (스냅 메뉴 목록)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 스물두 번째 함수 (FR-206 그리드 스냅 메뉴)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	스냅 메뉴 선택을 상태에 담는 배선을 진행합니다

1. 하는 일

스냅 메뉴에 넣을 **분할 목록**을 돌려줍니다. `DIVISION_DEN`의 여섯 분할(1/1~1/32)마다 **보통**과 **셋잇단** 두 줄을 만들어, 모두 12줄입니다.

- 셋잇단은 이름 뒤에 `T`를 붙입니다(`'1/16T'`).
- `ticks`는 `snapStepTicks()`로 구합니다(보통 `PPQ×4÷n`, 셋잇단 그 ÷3).
- 순서: 큰 분할부터(1/1), 각 분할은 보통 → 셋잇단.

라벨	division	triplet	ticks
1/1	1/1	false	3840
1/1T	1/1	true	1280
1/2	1/2	false	1920
1/2T	1/2	true	640
…	…	…	…
1/32	1/32	false	120
1/32T	1/32	true	40

2. 코드

```
export function snapOptions() {
  const out = [];
  for (const division of Object.keys(DIVISION_DEN)) {
    out.push({ division, triplet: false, label: division, ticks: snapStepTicks(division, false) });
    out.push({ division, triplet: true, label: division + 'T', ticks: snapStepTicks(division, true) });
  }
  return out;
}
```

최신 문법으로 쓰면 (공부용)

```
export const snapOptions = () =>
  Object.keys(DIVISION_DEN).flatMap((division) => [
    { division, triplet: false, label: division, ticks: snapStepTicks(division, false) },
```

```
    { division, triplet: true, label: `${division}T`, ticks: snapStepTicks(division, true) },
  ]);
```

최신 문법	실제 코드	뜻
`flatMap`	`for` + `push` 두 번	같은 결과입니다. 두 줄을 이어 붙이는 모양이 눈에 보이게 반복으로 씁니다

3. 왜 이렇게

선택	이유
`DIVISION_DEN`을 재사용	분할 이름·분모를 이미 한 곳(116)에서 정의했습니다. 새 목록을 만들면 둘이 어긋날 수 있습니다
보통 + 셋잇단을 한 줄씩	FR-206이 "1/1~1/32, 셋잇단 포함"을 요구합니다. 메뉴에서 바로 고를 수 있게 둘 다 넣습니다
`ticks`까지 함께	메뉴가 tick 값을 바로 알아 끌기 코드에 넘길 수 있습니다. 계산을 메뉴마다 반복하지 않습니다
`label`을 따로	화면에 보일 이름(`'1/16T'`)과 값(`division`·`triplet`)을 나눠, 보이는 글자와 동작을 독립시킵니다
호출마다 새 배열	메뉴를 다시 그릴 때 이전 배열을 건드리지 않습니다

4. 시험 (Node에서 직접 실행, 9개 통과)

확인	결과
줄 수 12 · 순서(보통·셋잇단 교대)	✓
보통 tick [3840,1920,960,480,240,120]	✓
셋잇단 tick [1280,640,320,160,80,40]	✓
`triplet` 표시 · `division` 값	✓
첫 줄·마지막 줄 객체	✓
호출마다 새 배열	✓

JS 137 — `setSnapDivision()` · `getSnapDivision()` (스냅 분할 저장·조회)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 스물세 번째 함수 (FR-206 그리드 스냅)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	고른 스냅을 실제 끌기에 쓰는 연결(선택값 → 스냅 단위)을 진행합니다

1. 하는 일

Snap 메뉴에서 고른 **분할과 셋잇단 여부**를 담아 두고(`setSnapDivision`), 나중에 꺼냅니다(`getSnapDivision`). 짝으로 움직이는 한 쌍이라 함께 둡니다.

- 담기 전에 이름을 `divisionTicks()`로 확인합니다. 없는 이름이면 **오류를 던지고 담지 않습니다**(이전 값 유지).
- 기본값은 `'1/16'`(보통).
- 조회는 지금 값을 새 객체로 돌려줍니다(사본).

하는 일	부르는 법	결과/반환
기본값 조회	`getSnapDivision()`	`{division:'1/16', triplet:false}`
1/4 담기	`setSnapDivision('1/4')`	`{division:'1/4', triplet:false}`
1/8T 담기	`setSnapDivision('1/8', true)`	`{division:'1/8', triplet:true}`
없는 값	`setSnapDivision('1/3')`	오류, 이전 값 유지

2. 코드

```
let snapDivision = '1/16';
let snapTriplet = false;

export function setSnapDivision(division, triplet = false) {
  divisionTicks(division);
  snapDivision = division;
  snapTriplet = triplet ? true : false;
  return { division: snapDivision, triplet: snapTriplet };
}

export function getSnapDivision() {
  return { division: snapDivision, triplet: snapTriplet };
}
```

최신 문법으로 쓰면 (공부용)

```
let state = { division: '1/16', triplet: false };
```

```
export const setSnapDivision = (division, triplet = false) => {
  divisionTicks(division);
  state = { division, triplet: Boolean(triplet) };
  return { ...state };
};

export const getSnapDivision = () => ({ ...state });
```

최신 문법	실제 코드	뜻
한 객체 `state`	변수 두 개	같은 뜻입니다. 값이 둘뿐이라 나눠 두면 읽기 쉬워 그대로 씁니다
`Boolean(triplet)`	`triplet ? true : false`	같은 결과입니다
`{ ...state }`	새 객체 리터럴	둘 다 사본을 돌려줍니다

3. 왜 이렇게

선택	이유
담기 전 이름 검사	오타나 없는 분할을 담으면 나중에 끌기에서 조용히 틀린 값이 됩니다. 담는 순간 막습니다
검사 실패 시 이전 값 유지	잘못 골랐다고 스냅이 사라지면 안 됩니다. 이전 선택을 그대로 둡니다
`triplet`을 boolean으로	`1` 같은 값이 들어와도 `true`로 맞춥니다. 나중 비교가 어긋나지 않습니다
조회가 사본 반환	돌려받은 객체를 고쳐도 저장값이 안 바뀝니다. 상태를 밖에서 망가뜨리지 못합니다
두 함수를 함께	저장과 조회는 같은 값 하나를 다룹니다. 따로 두면 어긋나므로 짝으로 봅니다

4. 시험 (Node에서 직접 실행, 12개 통과)

모듈 상태라 순서대로 이어서 확인했습니다.

확인	결과
기본값 `{1/16, false}`	✓
담기 반환값 · 담은 뒤 조회	✓
셋잇단 담기 · 조회	✓
`triplet` 생략은 보통 · 참 같은 값도 boolean	✓
없는 분할·빈 이름 → 오류	✓
오류 뒤 이전 값 유지	✓
조회는 사본(매번 새 객체)	✓

JS 138 — `divisionBars()` (고른 분할 → 마디 단위)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 스물네 번째 함수 (FR-206 그리드 스냅 연결)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	이 값을 `snap`·`snapUnit` 에 물리는 배선을 진행합니다

1. 하는 일

Snap 메뉴에서 고른 분할을 **마디 단위**로 바꿉니다. 끌기 코드(`snap`·`snapUnit`)가 마디 단위를 쓰므로 그 단위로 맞춰 줍니다.

- `divisionTicks()`(tick 길이)를 `TICKS_PER_BAR`(3840)로 나눕니다. 4/4에서 1/n 음표 = 1/n 마디입니다.
- 셋잇단이면 `tripletTicks()`(÷3)를 나눕니다.

분할	tick	마디
1/1	3840	1
1/2	1920	0.5
1/4	960	0.25
1/8	480	0.125
1/16	240	0.0625
1/32	120	0.03125
1/4T	320	0.0833… (=0.25/3)
1/16T	80	0.0208… (=1/48)

2. 코드

```
export function divisionBars(division, triplet = false) {
  return snapStepTicks(division, triplet) / TICKS_PER_BAR;
}
```

최신 문법으로 쓰면 (공부용)

```
export const divisionBars = (division, triplet = false) =>
  divisionTicks(division) / 3 ** Number(triplet) / TICKS_PER_BAR;
```

최신 문법	실제 코드	뜻
`3 ** Number(triplet)`	`snapStepTicks()` 안의 분기	같은 값입니다. 셋잇단 판단을 이미 있는 `snapStepTicks()`에 맡겨 한 곳에서만 다룹니다

3. 왜 이렇게

선택	이유
tick을 거쳐 마디로	`divisionTicks()`는 이미 검증·계산이 끝난 함수입니다. 이름 검사와 셋잇단 처리를 다시 만들지 않습니다
`snapStepTicks()` 재사용	칸+셋잇단을 한 값으로 만드는 규칙(119)이 이미 있습니다. 같은 규칙을 두 번 쓰지 않습니다
나누는 값으로 `TICKS_PER_BAR`	4/4 기준 3840입니다. 마디당 tick 수를 바꿔야 할 때 이 상수 한 곳만 고치면 됩니다
반환은 마디 단위	끌기 코드가 마디로 계산합니다. 여기서 tick으로 돌려주면 부르는 쪽마다 또 나눠야 합니다
이름 오류는 그대로 전달	`divisionTicks()`가 던진 오류를 감싸지 않고 그대로 올립니다. 원인이 한 곳에 남습니다

4. 시험 (Node에서 직접 실행, 11개 통과)

확인	결과
1/1~1/32 → 1, 0.5, 0.25, 0.125, 0.0625, 0.03125	✓
1/4T → 0.25/3 · 1/16T → 1/48	✓
`triplet` 생략은 보통과 같음	✓
없는 분할·빈 이름 → 오류	✓

JS 139 — `snapUnitNow()` (스냅 방식 하나로 모으기)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 스물다섯 번째 함수 (FR-206 — 고른 분할을 끌기에 연결)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	Snap 메뉴에 분할 목록을 그리고 클릭을 잇는 배선을 진행합니다

1. 하는 일

스냅 방식을 **한 값(마디 단위)** 으로 모읍니다. 끌기 코드는 이 값 하나만 봅니다.

방식	뜻	스냅 한 칸
`'off'`	격자 무시	1틱 (`TICK`)
`'division'`	내가 고른 분할	`divisionBars(고른 값)`
`'line'`	확대에 따라 자동	`getSnap()`

- Alt(`free`)를 누른 채 끌면 방식과 무관하게 **1틱**입니다.

```
snapUnitNow(free)
   ├ Alt 이거나 'off'  → TICK (1틱)
   ├ 'division'        → divisionBars(고른 분할)   ← FR-206
   └ 그 밖('line')      → getSnap()  (확대에 따라 자동)
```

배선: 스냅 한 칸을 구하던 `snapUnit`이 이제 `snapUnitNow`를 부르고, `setSnapDivision`으로 분할을 고르면 방식이 `'division'`이 됩니다.

2. 코드

```
export function snapUnitNow(free = false) {
  if (free || snapMode === 'off') return TICK;
  if (snapMode === 'division') return divisionBars(snapDivision, snapTriplet);
  return getSnap();
}

const snapUnit = (free) => snapUnitNow(free);
```

`setSnapDivision`에 한 줄을 더했습니다.

```
export function setSnapDivision(division, triplet = false) {
  divisionTicks(division);
  snapDivision = division;
  snapTriplet = triplet ? true : false;
  snapMode = 'division';
```

```
  return { division: snapDivision, triplet: snapTriplet };
}
```

최신 문법으로 쓰면 (공부용)

```
export const snapUnitNow = (free = false) =>
  free || snapMode === 'off' ? TICK
  : snapMode === 'division' ? divisionBars(snapDivision, snapTriplet)
  : getSnap();
```

최신 문법	실제 코드	뜻
중첩 삼항	`if` 세 줄	같은 결과입니다. 세 경우가 눈에 또렷하게 보이도록 `if`로 폈습니다

3. 왜 이렇게

선택	이유
한 함수에서 셋을 판단	끌기 코드가 `snapMode`를 곳곳에서 따지면 규칙이 흩어집니다. 한 곳에서만 정합니다
Alt 최우선	Alt는 "지금 이 순간 격자를 무시"라는 뜻입니다. 어떤 방식이든 앞서야 합니다
분할을 고르면 방식 전환	별도 "분할 모드"를 켜는 단계를 없앱니다. 고르는 순간이 곧 전환입니다
`'line'`은 그대로 자동	기존 확대 연동(Line) 동작을 건드리지 않습니다. Line·Off 버튼은 예전처럼 동작합니다
`snapUnit`은 얇게 유지	기존 호출부(`snap`·`floorSnap`)를 그대로 두려고 이름만 남기고 위임합니다

4. 시험 (Node에서 직접 실행, 11개 통과)

브라우저 없이 돌리려고 `document`를 가짜로 채워 넣고 불러왔습니다.

확인	결과
처음엔 `line` → `SNAP`(0.25)	✓
`off` → `TICK`	✓
분할을 고르면 방식이 `division` · 1/8 → 0.125 · 1/16T → 1/48	✓
Alt면 방식과 무관하게 1틱	✓
Line으로 돌아오면 자동(`SNAP`)	✓

JS 140 — Snap 메뉴 배선 (분할 목록 만들기·클릭 연결)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`, `studionx/frontend/studioNx.html`
작업	S1-06 타임라인: 스물여섯 번째 함수 (FR-206 스냅 메뉴 배선)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	트랙 추가·삭제 UI(FR-207)와 클립 배치 배선을 이어서 진행합니다

1. 하는 일

Snap 메뉴에 **분할 목록(1/1~1/32 · 셋잇단)** 을 만들고, 클릭을 `setSnapDivision`·`setSnapMode`에 잇습니다. 버튼 글자와 체크 표시도 지금 방식에 맞춰 다시 씁니다.

- `snapOptions()` 목록을 메뉴에 그립니다(12줄: 1/1, 1/1T, ..., 1/32T).
- 분할을 누르면 `setSnapDivision`, Line·Off를 누르면 `setSnapMode`를 부릅니다.
- 버튼: `Snap: Line ▾` · `Snap: Off ▾` · `Snap: 1/16T ▾` 처럼 바뀝니다.
- 여러 번 초기화해도 목록이 두 번 만들어지지 않습니다(이미 있으면 건너뜀).

```
Snap ▾ 클릭
   ├ Line        → setSnapMode('line')      → 버튼 'Snap: Line ▾'
   ├ Off         → setSnapMode('off')       → 버튼 'Snap: Off ▾'
   └ 1/16T       → setSnapDivision('1/16', true) → 버튼 'Snap: 1/16T ▾'
```

2. 코드

`sequencer-arrange.js`

```
function buildSnapDivisions(menu) {
  if (menu.querySelector('[data-division]')) return;
  const note = menu.querySelector('#snap-note');
  const frag = document.createDocumentFragment();
  for (const opt of snapOptions()) {
    const li = document.createElement('li');
    const b = document.createElement('button');
    b.type = 'button';
    b.setAttribute('role', 'menuitemradio');
    b.setAttribute('aria-checked', 'false');
    b.dataset.division = opt.division;
    b.dataset.triplet = String(opt.triplet);
    b.textContent = opt.label;
    li.appendChild(b);
    frag.appendChild(li);
  }
  if (note) menu.insertBefore(frag, note);
  else menu.appendChild(frag);
}
```

```
function updateSnapButton() {
  const btn = document.getElementById('snap');
  if (!btn) return;
  if (snapMode === 'off') btn.textContent = 'Snap: Off ▾';
  else if (snapMode === 'division') btn.textContent = `Snap: ${snapDivision}${snapTriplet ? 'T' : ''} ▾
`;
  else btn.textContent = 'Snap: Line ▾';
}

function refreshSnapMenu() {
  const menu = document.getElementById('menu-snap');
  if (!menu) return;
  menu.querySelectorAll('[data-snap]').forEach((b) => b.setAttribute('aria-checked', String(b.dataset.sn
ap === snapMode)));
  menu.querySelectorAll('[data-division]').forEach((b) => {
    const on = snapMode === 'division' && b.dataset.division === snapDivision && (b.dataset.triplet ===
'true') === snapTriplet;
    b.setAttribute('aria-checked', String(on));
  });
}
```

클릭 연결 (initSnapMenu 안)

```
menu.addEventListener('click', (e) => {
  const item = e.target.closest('[data-snap],[data-division]');
  if (!item) return;
  if (item.dataset.division) setSnapDivision(item.dataset.division, item.dataset.triplet === 'true');
  else setSnapMode(item.dataset.snap);
  close();
});
```

`setSnapMode`·`setSnapDivision` 은 상태를 바꾼 뒤 `updateSnapButton()`·`refreshSnapMenu()`·`notifyGridSettings()` 를 부릅니다.

`studioNx.html` (메뉴 골격에 구분 줄과 자리표 추가)

```
<li class="sq-dropdown__note" role="separator">Division</li>
<li class="sq-dropdown__note" id="snap-note">Hold Alt while dragging to ignore the grid</li>
```

최신 문법으로 쓰면 (공부용)

```
const frag = snapOptions().map((opt) => {
  const b = document.createElement('button');
  Object.assign(b.dataset, { division: opt.division, triplet: String(opt.triplet) });
  b.textContent = opt.label;
  return b;
});
menu.append(...frag);
```

최신 문법	실제 코드	뜻
`map` · `append(...)`	`for` + `frag.appendChild`	같은 결과입니다. `<li>`로 감싸는 단계가 보이도록 반복으로 씁니다

최신 문법	실제 코드	뜻
`Object.assign(b.dataset, …)`	`b.dataset.division = …`	같은 뜻입니다

3. 왜 이렇게

선택	이유
목록을 `snapOptions()`로 그림	분할·이름·셋잇단을 한 곳(136)에서 정의했습니다. 화면에 손으로 12줄을 적으면 어긋납니다
두 번 만들지 않음	창 크기 변화 등으로 다시 초기화해도 메뉴가 겹쳐 늘지 않습니다
클릭 한 곳에서 판단	Line·Off와 분할을 한 핸들러에서 나눠 처리합니다. 항목마다 따로 붙이지 않습니다
버튼·체크를 다시 씀	지금 고른 값이 어디서 바뀌든 화면이 맞춰집니다. 한 곳(`updateSnapButton`·`refreshSnapMenu`)에서만 그립니다
HTML은 골격만	구분 줄과 자리표(`#snap-note`)만 두고 항목은 JS가 채웁니다. 값을 바꿔도 HTML을 안 고칩니다

4. 시험

(1) Node 단위 시험(11개 통과) — `document`를 가짜로 채워 상태·전환을 확인했습니다(Line/Off/division, Alt 우선, Line 복귀).

(2) 브라우저 확인(Chrome headless + CDP, 실제 `studioNx.html`)

확인	결과
분할 줄 수 = 12 (여러 번 초기화해도 24 안 됨)	✓
라벨 순서 1/1, 1/1T, …, 1/32T	✓
버튼 글자: 열기 전 `Snap: Line ▾` → 1/16T 클릭 `Snap: 1/16T ▾` → Line 클릭 `Snap: Line ▾`	✓
1/16T 클릭 뒤 방식 `'division'` · 스냅 한 칸 1/48	✓
체크 표시 한 줄만, Line으로 복귀 시 Line 체크	✓

검토용 화면: `docs/ai-notes/images/javascript_s1-06/s1-06_snap-menu.png` (1/8T를 고른 채 메뉴를 연 모습).

JS 141 — `deleteTrack()` (트랙 삭제)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`, `studionx/frontend/static/css/pages/studio.css`
작업	S1-06 타임라인: 스물일곱 번째 함수 (FR-207 트랙 삭제)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	트랙 순서 드래그(그립) 배선과 이름·색 변경을 이어서 진행합니다

1. 하는 일

트랙 머리의 **× 버튼**을 누르면 그 트랙 줄을 지우고 남은 번호를 다시 매깁니다.

```
1. Kick   2. Bass   3. Lead        ← 2번의 × 클릭
1. Kick             2. Lead        ← 줄을 지우고 번호를 다시 매김
```

- MASTER는 지우지 않습니다(맨 아래 고정 줄). MIDI · 오디오 · 리턴 · 그룹은 지웁니다.
- × 는 평소엔 숨겨 두고, 트랙 머리에 마우스를 올리거나 키보드 포커스가 닿으면 보입니다.

하는 일	부르는 것	결과
× 클릭	`deleteTrack(track, onChange)`	줄 삭제 + 번호정리 + 알림
MASTER ×	없음(버튼 안 만듦)	MASTER 못 지움
+ Add Track	`addTrack()`	새 줄에 × 포함, 번호 다시 매김

2. 코드

`sequencer-arrange.js`

```
export function deleteTrack(track, notify = () => {}) {
  if (!track || track.classList.contains('sq-track--master')) return null;
  const trackId = track.dataset.trackId;
  track.remove();
  renumberTracks(notify);
  notify({ type: 'delete', trackId });
  return trackId;
}

function ensureTrackDelete(root) {
  root.querySelectorAll('.sq-track').forEach((track) => {
    if (track.classList.contains('sq-track--master')) return;
    const head = track.querySelector('.sq-track__head');
    if (!head || head.querySelector('.sq-track__del')) return;
    head.append(trackDeleteButton());
  });
}
```

```
function trackDeleteButton() {
  const b = document.createElement('button');
  b.type = 'button';
  b.className = 'sq-track__del';
  b.setAttribute('aria-label', 'Delete track');
  b.setAttribute('title', 'Delete track');
  b.textContent = '×';
  return b;
}
```

클릭 연결 (initTrackButtons 안)

```
if (b.matches('.sq-track__del')) { deleteTrack(track, onChange); return; }
```

`createTrack()`·`addTrack()`의 머리 HTML에도 × 버튼을 넣었습니다.

`studio.css`

```
.sq-track__del {
  flex: none; width: 1.1rem; height: 1.1rem; display: grid; place-items: center;
  border: 1px solid var(--sq-line-2); font: 700 0.7rem/1 var(--font-text);
  color: var(--c-ink-soft); background: rgb(0 0 0 / 0.15); opacity: 0;
  transition: opacity 0.12s;
}
.sq-track__head:hover .sq-track__del,
.sq-track__del:focus-visible { opacity: 1; }
.sq-track__del:hover { color: var(--sq-rec); border-color: var(--sq-rec); }
.sq-track--master .sq-track__del { display: none; }
```

최신 문법으로 쓰면 (공부용)

```
export const deleteTrack = (track, notify = () => {}) => {
  if (!track || track.classList.contains('sq-track--master')) return null;
  const { trackId } = track.dataset;
  track.remove();
  renumberTracks(notify);
  notify({ type: 'delete', trackId });
  return trackId;
};
```

최신 문법	실제 코드	뜻
`const { trackId } = track.dataset`	`track.dataset.trackId`	같은 값입니다

3. 왜 이렇게

선택	이유
줄을 지운 뒤 번호정리	번호는 자리에서 나옵니다. 지우고 `renumberTracks()`를 부르면 빈 번호가 남지 않습니다
MASTER 보호	MASTER는 항상 맨 아래 필요한 줄입니다. 함수와 CSS 두 곳에서 막아 실수로 못 지웁니다
기존 줄엔 JS로 버튼 삽입	트랙 줄이 13개라 HTML을 13번 고치는 것보다, 한 곳(`ensureTrackDelete`)에서 한 번에 붙이는 편이 안전합니다

선택	이유
이미 있으면 건너뜀	다시 초기화해도 × 가 겹쳐 붙지 않습니다
× 는 hover 시에만	평소 화면이 깔끔합니다. 실수로 누를 위험도 줄입니다(M/A/S 옆이지만 숨겨 둠)
지운 뒤 알림	저장(S1-10)이 붙을 때 `{type:'delete'}`를 받아 처리합니다

4. 시험

(1) Node 단위 시험 — 모듈 적재만 확인(11개 그대로 통과).

(2) 브라우저 확인(Chrome headless + CDP, 실제 `studioNx.html`)

확인	결과
처음 번호 1~10 → 2번 × 클릭 뒤 1~9, 줄 수 11→10	✓
MASTER 줄에 × 없음(`masterHasDel` false) · `deleteTrack(master)` → null · MASTER 남음	✓
다시 초기화해도 × 수 안 늘어남(중복 방지)	✓
+ Add Track 뒤 새 줄(11. MIDI)에 × 있음	✓
× 기본 `opacity: 0`(숨김, hover 시 표시)	✓

검토용 화면: `docs/ai-notes/images/javascript_s1-06/s1-06_track-delete.png` (× 를 보이도록 한 모습).

JS 142 — 트랙 순서 드래그 배선 (`initTrackReorder`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 스물여덟 번째 작업 (FR-207 트랙 순서 드래그)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	트랙 이름·색 변경(FR-207)과 오디오 클립 배치 배선을 이어서 진행합니다

1. 하는 일

트랙 머리의 ⠿ **그립**을 위아래로 끌어 순서를 바꿉니다. 놓으면 `reorderItems()`로 새 순서를 만들고 DOM을 다시 붙인 뒤, `renumberTracks()`로 번호를 다시 매깁니다.

```
1. Lead   ⠿        ← 0번 그립을 아래로 끌어 2번 자리로
2. Strum
3. Drums
─────────
1. Strum
2. Drums
3. Lead           ← 옮기고 번호 다시 매김
```

- 끄는 동안 그 트랙에 `is-reorder-dragging`, 놓일 자리 트랙에 `is-reorder-target`(CSS가 표시)을 붙입니다.
- 그룹 줄(`Low End GROUP`)처럼 번호 없는 줄은 배열에 그대로 있고, 번호정리에서 건너뜁니다.
- 새로 만든 트랙(`createTrack`·`addTrack`)에도 그립을 넣어 새 트랙도 끌 수 있습니다.

2. 코드

```
function initTrackReorder({ lanes, onChange }) {
  const tracksEl = lanes.querySelector('#tracks');
  if (!tracksEl) return;
  let drag = null;
  const items = () => Array.from(tracksEl.children).filter((el) => el.classList.contains('sq-track'));
  const clearMarks = () => items().forEach((el) => el.classList.remove('is-reorder-target'));

  const onMove = (e) => {
    if (!drag) return;
    const list = items();
    let at = 0;
    for (const el of list) { const r = el.getBoundingClientRect(); if (e.clientY > r.top + r.height / 2)
at += 1; }
    const to = Math.max(0, Math.min(list.length - 1, drag.from < at ? at - 1 : at));
    drag.to = to;
    clearMarks();
    const target = list[to];
    if (target && target !== drag.track) target.classList.add('is-reorder-target');
  };
```

```
  function end(commit) {
    if (!drag) return;
    const d = drag; drag = null; cleanup();
    d.track.classList.remove('is-reorder-dragging');
    clearMarks();
    if (!commit) return;
    const list = items();
    const next = reorderItems(list, d.from, d.to);
    let changed = false;
    for (let i = 0; i < next.length; i++) { if (next[i] !== list[i]) { changed = true; break; } }
    if (!changed) return;
    next.forEach((el) => tracksEl.append(el));
    renumberTracks(onChange);
    onChange({ type: 'reorder', trackId: d.track.dataset.trackId, from: d.from, to: d.to });
  }

  lanes.addEventListener('pointerdown', (e) => {
    const grip = e.target.closest('.sq-grip');
    if (!grip || e.button !== 0) return;
    const track = grip.closest('.sq-track');
    if (!track || !tracksEl.contains(track)) return;
    e.preventDefault();
    const list = items();
    drag = { track, from: list.indexOf(track), to: list.indexOf(track) };
    track.classList.add('is-reorder-dragging');
    document.addEventListener('pointermove', onMove);
    document.addEventListener('pointerup', onUp);
    document.addEventListener('pointercancel', onCancel);
  });
}
```

(`cleanup`·`onUp`·`onCancel`은 위 끝맺음과 함께 붙습니다.) **최신 문법으로 쓰면** (공부용)

```
const items = () => [...tracksEl.children].filter((el) => el.classList.contains('sq-track'));
```

최신 문법	실제 코드	뜻
스프레드 `[...children]`	`Array.from(children)`	같은 결과입니다

3. 왜 이렇게

선택	이유
`to` 계산에 `from < at ? at-1 : at`	끄는 항목을 빼면 아래가 한 칸 당겨집니다. 그만큼 보정해야 손이 가리킨 자리에 놓입니다
그립에서만 시작	트랙 머리 빈 곳을 눌러도 드래그가 시작되지 않습니다. 좌우 이동(pan)·클립 끌기와 겹치지 않습니다
`reorderItems()` 재사용	순서 바꾸기 규칙(131)을 이미 시험했습니다. 화면 코드는 그 결과를 DOM에 붙이기만 합니다
안 바뀌면 건너뜀	제자리에 놓으면 번호정리도 알림도 하지 않습니다. 불필요한 저장을 막습니다
놓기 전에는 안 바꿈	미리보기(표시)만 하고, 실제 순서는 놓을 때 한 번 바꿉니다. 되돌리기가 쉽습니다

4. 시험

(1) Node 단위 시험 — 모듈 적재 확인(11개 통과). `reorderItems` 자체는 131에서 15개 통과.

(2) 브라우저 확인(Chrome headless + CDP, 실제 `studioNx.html`) — 그립에 `pointerdown` → `pointermove`(아래로) → `pointerup` 을 흉내 냈습니다.

확인	결과
끄는 동안 `is-reorder-dragging`, 놓일 자리 `is-reorder-target` 표시	✓
0번(Lead)을 2번 자리로 → 순서 `Lead,Strum,Drums → Strum,Drums,Lead`	✓
놓은 뒤 번호가 자리대로 다시 매겨짐(1.Strum·2.Drums·3.Lead)	✓
그룹 줄(번호 없음)은 자리 유지·번호정리에서 건너뜀	✓
놓은 뒤 표시(`is-reorder-*`)가 지워짐	✓

JS 143 — 클립 오른쪽 길이 조절 배선 (`initClipDrag` 확장)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 스물아홉 번째 작업 (FR-208 클립 오른쪽 길이 조절)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	왼쪽 끝 트리밍(`trimClipLeft`, offset_tick) 배선을 이어서 진행합니다

1. 하는 일

클립 **오른쪽 끝 6px 안쪽**을 잡으면 옮기기가 아니라 **길이 조절**이 됩니다. 끌면 격자에 맞춰 길이가 늘고 주는데, **최소 한 칸**보다 짧아지지 않습니다(더 끌어도 삭제되지 않고 멈춤).

```
[■■■■ Kick        ]      ← 오른쪽 끝을 오른쪽으로 끌기
[■■■■■■■■ Kick    ]      ← 격자에 맞춰 길어짐 (min = 한 칸)
[■ Kick           ]      ← 왼쪽으로 끝까지 끌어도 한 칸에서 멈춤
```

- 잡은 자리: 클립 끝에서 6px 안쪽(`EDGE_PX`)이면 조절, 그 밖은 기존대로 옮기기.
- 옮기기는 여러 클립을 함께 옮기지만, 길이 조절은 **잡은 클립 하나만** 바뀝니다.
- Alt 를 누른 채 끌면 격자를 무시하고 1틱까지 줄일 수 있습니다(`snapUnit` 규칙 그대로).
- 끄는 동안 클립에 `is-resizing`(CSS가 좌우 화살표 커서·손잡이 색을 켬)을 붙입니다.

2. 코드

`EDGE_PX` 상수 (파일 위쪽)

```
/** 클립 끝에서 이만큼(px) 안쪽을 잡으면 옮기기가 아니라 길이 조절로 봄 (FR-208) */
const EDGE_PX = 6;
```

`initClipDrag` 안 — 잡았을 때 모드 정하기

```
const rect = clip.getBoundingClientRect();
const onRight = rect.right - e.clientX <= EDGE_PX;   // 오른쪽 끝 손잡이를 잡았나
drag = {
  clip, multi, wasSelected,
  mode: onRight ? 'resize' : 'move',   // 오른쪽 끝 = 길이 조절, 그 밖 = 옮기기
  id: e.pointerId, x: e.clientX, scrollLeft: scroll.scrollLeft,
  group: selected().map((c) => ({
    clip: c,
    start: parseFloat(c.style.getPropertyValue('--s')) || 0,
    length: parseFloat(c.style.getPropertyValue('--l')) || 1,
  })),
  moved: false,
};
drag.primary = drag.group.find((g) => g.clip === clip);
```

끄는 중 — 클립 오른쪽 길이를 조절할 때 실행되는 코드

```
if (drag.mode === 'resize') {
  const stepTicks = barsToTicks(snapUnit(e.altKey));            // Alt = 격자 무시(1틱)
  const deltaTicks = barsToTicks(dx / barWidth(lanes));
  const lenTick = resizeClipLength(barsToTicks(drag.primary.length), deltaTicks, stepTicks);
  drag.primary.clip.style.setProperty('--l', round(ticksToBars(lenTick)));
  edgeScroll(scroll, e.clientX);
  return;
}
```

놓을 때 — 알림(`onChange`)

```
if (mode === 'resize') {
  clip.classList.remove('is-resizing');
  const length = parseFloat(clip.style.getPropertyValue('--l'));
  if (length !== drag.primary.length) onChange({ type: 'resize', clip, trackId: clip.closest('.sq-trac
k').dataset.trackId, start: drag.primary.start, length });
} else { /* 기존 move 처리 */ }
```

최신 문법으로 쓰면 (공부용)

```
const mode = rect.right - e.clientX <= EDGE_PX ? 'resize' : 'move';
```

최신 문법	실제 코드	뜻
`const mode = cond ? a : b`	같은 코드	여기서는 그대로 씁니다

3. 왜 이렇게

선택	이유
화면(마디)과 함수(tick) 단위를 변환	화면 클립은 `--s`·`--l` 이 마디 단위입니다. 검증된 함수 121~125 는 tick 단위라 `barsToTicks`·`ticksToBars`(115·116)로 오갑니다
`resizeClipLength()` 재사용	스냅·최소 한 칸 규칙을 이미 시험했습니다(122). 화면 코드는 결과만 `--l` 에 씁니다
끝 6px 만 조절	옮기려고 잡았는데 길이가 바뀌면 답답합니다. 손잡이 폭(6px)을 그대로 씁니다
Alt 로 1틱까지	스냅 규칙과 같게 맞춰, 미세 조절이 필요할 때 씁니다
잡은 클립만 조절	여러 클립의 길이를 한 번에 바꾸는 것은 규칙이 복잡합니다. 지금은 하나만 합니다

4. 시험

(1) Node 단위 시험 — 모듈 적재 확인(11개 통과). `resizeClipLength` 자체는 122에서 14개 통과.

(2) 브라우저 확인(Chrome headless + CDP 실제 마우스 이벤트, 실제 `studioNx.html`) — 클립 오른쪽 끝을 잡고 끌었습니다.

확인	결과
오른쪽 끝을 오른쪽으로 끌기 → 길이 4 → 7 마디(격자에 맞춰 증가)	✓
끄는 동안 `is-resizing` 표시	✓

확인	결과
왼쪽으로 끝까지 끌기 → 최소 한 칸(1 마디)에서 멈추고 삭제되지 않음	✓
가운데를 끌면 예전대로 옮기기(시작 0 → 5, 길이 그대로), `is-dragging` 표시	✓

JS 144 — 클립 왼쪽 끝 트리밍 배선 (`initClipDrag` 확장)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 서른 번째 작업 (FR-208 오디오 트리밍, `offset_tick`)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	클립 복제(Ctrl+D, `duplicateClipStart`) 배선을 이어서 진행합니다

1. 하는 일

클립 **왼쪽 끝 6px 안쪽**을 잡으면 **트리밍**이 됩니다. 시작·길이·오프셋 세 값이 함께 움직여, 오디오는 화면에서 소리가 제자리에 고정된 채 잘립니다.

```
[■■■■■■■ Vocal ]      ← 왼쪽 끝을 오른쪽으로 (앞을 잘라냄)
    [■■■■ Vocal ]      ← 시작 +3, 길이 -3, offset +3 (원본에서 3만큼 더 건너뜀)
```

- 줄이기: 오른쪽 조절과 같은 규칙. 최소 한 칸이고, Alt 면 1틱까지.
- 늘리기: **원본 처음**과 **마디 0** 에 닿을 때까지만. MIDI(원본 없음)는 마디 0 까지 자유롭게, 오프셋은 늘 0.
- 오디오 클립(`sq-clip--audio`)은 `start - offset` 을 원본 처음으로 봐서 그보다 왼쪽으로는 못 갑니다.
- 끄는 동안 `is-resizing`(오른쪽 조절과 같은 표시)을 붙입니다.

클립	늘리기 한계	오프셋
오디오	`start - offset` (`sourceStartTick`)	자른 만큼 증가 (`--o`)
MIDI	마디 0	항상 0

2. 코드

잡았을 때 — 모드와 원본 처음

```
const onLeft = e.clientX - rect.left <= EDGE_PX;    // 왼쪽 끝 손잡이 = 트리밍
const onRight = rect.right - e.clientX <= EDGE_PX;  // 오른쪽 끝 손잡이 = 길이 조절
drag = {
  clip, multi, wasSelected,
  mode: onRight ? 'resize' : onLeft ? 'trim' : 'move',
  id: e.pointerId, x: e.clientX, scrollLeft: scroll.scrollLeft,
  group: selected().map((c) => ({
    clip: c,
    start: parseFloat(c.style.getPropertyValue('--s')) || 0,
    length: parseFloat(c.style.getPropertyValue('--l')) || 1,
    offset: parseFloat(c.style.getPropertyValue('--o')) || 0,
  })),
  moved: false,
};
drag.primary = drag.group.find((g) => g.clip === clip);
// 오디오는 원본 처음이 타임라인 어디인지(start - offset)를 알아야 늘리기 한계를 지킴. MIDI 는 원본이 없어
```

```
null
drag.sourceStartTick = clip.classList.contains('sq-clip--audio')
  ? barsToTicks(drag.primary.start) - barsToTicks(drag.primary.offset)
  : null;
```

끄는 중 — 클립을 트리밍할 때 실행되는 코드

```
if (drag.mode === 'trim') {
  const stepTicks = barsToTicks(snapUnit(e.altKey));
  const deltaTicks = barsToTicks(dx / barWidth(lanes));
  const res = trimClipLeft(barsToTicks(drag.primary.start), barsToTicks(drag.primary.length), deltaTick
s, stepTicks, drag.sourceStartTick);
  drag.primary.clip.style.setProperty('--s', round(ticksToBars(res.start)));
  drag.primary.clip.style.setProperty('--l', round(ticksToBars(res.length)));
  drag.primary.clip.style.setProperty('--o', round(ticksToBars(res.offset)));
  edgeScroll(scroll, e.clientX);
  return;
}
```

놓을 때 — 알림(`onChange`)

```
} else if (mode === 'trim') {
  clip.classList.remove('is-resizing');
  const start = parseFloat(clip.style.getPropertyValue('--s'));
  const length = parseFloat(clip.style.getPropertyValue('--l'));
  const offset = parseFloat(clip.style.getPropertyValue('--o')) || 0;
  if (start !== drag.primary.start || length !== drag.primary.length) onChange({ type: 'trim', clip, tra
ckId: clip.closest('.sq-track').dataset.trackId, start, length, offset });
}
```

최신 문법으로 쓰면 (공부용)

```
drag.sourceStartTick = clip.classList.contains('sq-clip--audio')
  ? barsToTicks(drag.primary.start) - barsToTicks(drag.primary.offset)
  : null;
```

최신 문법	실제 코드	뜻
`a ? b : c`	같은 코드	여기서는 그대로 씁니다

3. 왜 이렇게

선택	이유
`trimClipLeft()` 재사용	세 값(시작·길이·오프셋)을 함께 움직이는 규칙을 이미 시험했습니다(123). 화면은 결과만 씁니다
`start − offset` 로 원본 처음 구함	오디오는 앞을 잘라도 소리 위치가 고정돼야 합니다. 원본 기준점을 알아야 그 왼쪽으로 못 늘립니다
MIDI 는 `null`	원본 소스가 없어 마디 0 까지 자유롭게 늘려도 됩니다. 오프셋은 개념이 없어 0
오른쪽(143)과 같은 `EDGE_PX`	두 손잡이가 같은 폭이라 잡는 느낌이 일정합니다
오른쪽 조절 코드를 먼저 냄	같은 `initClipDrag` 안에서 경우(모드)만 늘리는 방식이라, 되돌리기 쉽고 충돌이 없습니다

4. 시험

(1) Node 단위 시험 — 모듈 적재 확인(11개 통과). `trimClipLeft` 자체는 123에서 통과.

(2) 브라우저 확인(Chrome headless + CDP 실제 마우스 이벤트, 실제 `studioNx.html`) — 클립 왼쪽 끝을 잡고 끌었습니다.

확인	결과
MIDI 왼쪽 끝을 오른쪽으로 → 시작 8 → 11, 길이 4 → 1(최소 한 칸), 오프셋 0 유지	✓
MIDI 왼쪽 끝을 왼쪽으로 → 시작 0(마디 0 한계), 길이 12, 오프셋 0	✓
오디오 왼쪽 끝을 오른쪽으로 → 시작 2 → 7, 길이 10 → 5, **오프셋 0 → 5** (원본에서 더 건너뜀)	✓
끄는 동안 `is-resizing` 표시(오른쪽 길이 조절과 같은 표시)	✓

JS 145 — 클립 복제 배선 (Ctrl+D, `duplicateClipStart`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 서른한 번째 작업 (FR-208 클립 복제, Ctrl+D)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	클립 분할(`splitClip`) 배선을 이어서 진행합니다

1. 하는 일

선택한 클립을 **Ctrl+D** 로 복제합니다. 복제본은 원본 **바로 오른쪽**(시작 + 길이)에 붙고, 만들어진 복제본이 선택됩니다. 다시 Ctrl+D 를 누르면 그 복제본이 또 오른쪽으로 복제돼 연속으로 이어집니다.

```
[■■■■ Lead ]                          ← Ctrl+D
[■■■■ Lead ][■■■■ Lead ]              ← 원본 오른쪽에 복제, 복제본이 선택됨
[■■■■ Lead ][■■■■ Lead ][■■■■ Lead ] ← 다시 Ctrl+D (연속)
```

- 시작 위치는 `duplicateClipStart(startTick, lengthTick)` = `start + length` 를 씁니다.
- 곡 끝을 넘지 않게 `totalBars` 안으로 당깁니다(`clamp`).
- 오디오 클립은 오프셋(`--o`)도 복사해서 같은 대목이 다시 들립니다.
- 입력칸(`input`·`textarea` 등)에 커서가 있거나 대화상자가 열려 있으면 동작하지 않습니다.

2. 코드

```
// Ctrl+D → 선택한 클립을 바로 오른쪽에 복제 (FR-208 복제). 새 복제본이 선택됨
document.addEventListener('keydown', (e) => {
  if (!(e.ctrlKey || e.metaKey) || e.key.toLowerCase() !== 'd' || e.target.closest?.('input, textarea, s
elect, [contenteditable]') || document.querySelector('dialog[open]')) return;
  const clips = [...lanes.querySelectorAll('.sq-clip.is-selected:not(.is-ghost)')];
  if (!clips.length) return;
  e.preventDefault();
  const copies = clips.map((c) => {
    const track = c.closest('.sq-track');
    const start = parseFloat(c.style.getPropertyValue('--s')) || 0;
    const length = parseFloat(c.style.getPropertyValue('--l')) || 1;
    const offset = parseFloat(c.style.getPropertyValue('--o')) || 0;
    const kind = c.classList.contains('sq-clip--audio') ? 'audio' : 'midi';
    const name = c.querySelector('.sq-clip__name').textContent.replace(/^[目⇥]\s*/, '').trim();
    const newStart = clamp(ticksToBars(duplicateClipStart(barsToTicks(start), barsToTicks(length))), 0,
totalBars(lanes) - length);
    const clip = createClip({ kind, name, start: newStart, length });
    if (offset) clip.style.setProperty('--o', offset);
    track.querySelector('.sq-track__lane').append(clip);
    onChange({ type: 'duplicate', clip, trackId: track.dataset.trackId, start: newStart, length, offset
});
    return clip;
```

```
  });
  clearSelection(lanes);
  copies.forEach((c) => c.classList.add('is-selected'));
});
```

최신 문법으로 쓰면 (공부용)

```
const clips = [...lanes.querySelectorAll('.sq-clip.is-selected:not(.is-ghost)')];
```

최신 문법	실제 코드	뜻
스프레드 `[...]`	`[...lanes.querySelectorAll(...)]`	같은 코드입니다(NamedNodeList 를 배열로)

3. 왜 이렇게

선택	이유
`duplicateClipStart()` 재사용	"원본 오른쪽에 붙인다"는 규칙을 이미 시험했습니다(125). 화면은 시작 위치만 받아 씁니다
복제본을 선택	연속 Ctrl+D 가 자연스럽게 이어집니다. 원본은 선택이 풀려 헷갈리지 않습니다
원본과 같은 트랙에 넣음	복제는 같은 자리에서 바로 쓰는 것이 보통입니다. 트랙을 옮기는 건 따로 끌면 됩니다
오프셋(`--o`) 복사	오디오는 잘라 쓴 대목이 있으므로, 복제본도 같은 대목에서 시작해야 합니다
곡 끝 `clamp`	마지막 클립을 복제해도 곡 밖으로 나가지 않습니다
입력칸·대화상자 예외	이름을 고치거나 창을 쓰는 중에 실수로 복제되지 않습니다 (`Ctrl+A` 와 같은 방식)

4. 시험

(1) Node 단위 시험 — 모듈 적재 확인(11개 통과). `duplicateClipStart` 자체는 125에서 통과.

(2) 브라우저 확인(Chrome headless + CDP 실제 키 이벤트, 실제 `studioNx.html`)

확인	결과
클립(Lead, 시작 0·길이 4) 선택 후 Ctrl+D → 클립 수 3 → 4, 복제본 시작 4(=0+4), 이름·길이 그대로	✓
복제본만 선택됨(selected 1개, 원본 풀림)	✓
다시 Ctrl+D → 클립 수 5, 새 복제본 시작 8(연속)	✓
오디오 클립 복제 → 오프셋 4 유지, 시작 8 → 12, 종류 audio	✓

JS 146 — 클립 분할 배선 (Slice 도구, `splitClip`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 서른두 번째 작업 (FR-208 클립 분할)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	FR-209 가로·세로 줌 배선을 이어서 진행합니다

1. 하는 일

Slice (C) 도구로 클립을 누르면 누른 자리를 격자에 맞춰 **둘로** 가릅니다. 두 토막이 각각 한 칸 이상일 때만 가르고, 너무 끝에 가까우면(한 칸 미만) 가르지 않습니다.

```
[■■■■ Lead ]                ← Slice 로 가운데 클릭
[■■ Lead ][■■ Lead ]        ← 두 토막 (각각 한 칸 이상일 때만)

[■■■■ Lead ]                ← 끝 3px 을 클릭 (한 칸 미만)
[■■■■ Lead ]                ← 가르지 않음
```

- 자른 자리는 격자에 맞추고, 왼쪽 토막은 `--l` 이 줄고 오른쪽 토막이 그 뒤에 새로 생깁니다.
- 오디오는 오른쪽 토막의 오프셋(`--o`)이 **왼쪽 길이만큼 커져** 원본에서 이어서 들립니다.
- Mute 된 클립을 자르면 오른쪽 토막도 Mute 를 이어받습니다.

클립	왼쪽 토막	오른쪽 토막
위치	시작 그대로	자른 지점
오프셋	그대로	원본 오프셋 + 왼쪽 길이

2. 코드

```
/** Slice: 누른 자리(격자)에서 둘로 — 두 토막이 각각 한 칸 이상일 때만 가름 (S1-06 FR-208) */
function sliceClip(clip, bar, free) {
  const s = parseFloat(clip.style.getPropertyValue('--s')) || 0;
  const l = parseFloat(clip.style.getPropertyValue('--l')) || 1;
  const offset = parseFloat(clip.style.getPropertyValue('--o')) || 0;
  const stepTicks = barsToTicks(snapUnit(free));   // Alt = 격자 무시(1틱)
  // 오디오는 원본 처음(start - offset)을 넘겨야 오른쪽 토막 오프셋이 이어짐. MIDI 는 null
  const source = clip.classList.contains('sq-clip--audio') ? barsToTicks(s) - barsToTicks(offset) : nul
l;
  const at = snap(bar, free);
  const res = splitClip(barsToTicks(s), barsToTicks(l), barsToTicks(at), stepTicks, source);
  if (!res) return;
  clip.style.setProperty('--s', round(ticksToBars(res.left.start)));
  clip.style.setProperty('--l', round(ticksToBars(res.left.length)));
  clip.style.setProperty('--o', round(ticksToBars(res.left.offset)));
  const right = createClip({ ...templateOf(clip), start: round(ticksToBars(res.right.start)), length: ro
```

```
und(ticksToBars(res.right.length)) });
  right.style.setProperty('--o', round(ticksToBars(res.right.offset)));
  if (clip.classList.contains('is-muted')) right.classList.add('is-muted');
  clip.after(right);
  onChange({ type: 'slice', clip, right, trackId: clip.closest('.sq-track').dataset.trackId, at: round(t
icksToBars(res.right.start)), offset: round(ticksToBars(res.right.offset)) });
}
```

최신 문법으로 쓰면 (공부용)

```
const right = createClip({ ...templateOf(clip), start: round(ticksToBars(res.right.start)), length: roun
d(ticksToBars(res.right.length)) });
```

최신 문법	실제 코드	뜻
`{ ...t, start, length }`	같은 코드	`templateOf(clip)` 결과에 시작·길이를 덮어씀

3. 왜 이렇게

선택	이유
`splitClip()` 재사용	자르기 규칙(스냅·최소 한 칸·오프셋 이어짐)을 이미 시험했습니다(124). 예전 프로토타입은 이 규칙을 손으로 다시 적어 오프셋 방향이 반대였습니다
최소 한 칸 검사	아주 짧은 토막이 생기면 고르기도 지우기도 어렵습니다. `splitClip` 이 `null` 을 돌려주면 그냥 무시
왼쪽 토막을 원래 클립에	새로 만드는 클립을 하나로 줄여, 이름·선택·Mute 등 원래 상태가 그대로 남습니다
`source = start - offset`	오디오가 원본에서 이어지려면 기준점이 필요합니다. 트리밍(144)과 같은 방식
Mute 이어받기	잘라도 소리 상태가 같아야 자연스럽습니다

4. 시험

(1) Node 단위 시험 — 모듈 적재 확인(11개 통과). `splitClip` 은 124에서 통과.

(2) 브라우저 확인(Chrome headless + CDP 실제 마우스 이벤트, 실제 `studioNx.html`) — Slice 도구(단축키 C)로 클립을 눌렀습니다.

확인	결과
Slice 도구 선택(`.sq` 클래스 `tool-slice`)	✓
MIDI(시작 0·길이 4) 가운데 클릭 → 클립 3 → 4, 왼쪽 0·2 / 오른쪽 2·2	✓
끝 3px 클릭(한 칸 미만) → 가르지 않음(4 → 4)	✓
오디오(시작 2·길이 10·오프셋 2) 가운데 클릭 → 왼쪽 2·5·2 / 오른쪽 7·5·**7**(= 2 + 5)	✓

JS 147 — 가로·세로 줌 배선 (`clampZoom` · `zoomScrollLeft` · `stepTrackHeight`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 서른세 번째 작업 (FR-209 가로·세로 줌)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	FR-210 오디오 클립 파형 배선을 이어서 진행합니다

1. 하는 일

가로 줌(Ctrl+휠, VIEW 메뉴 확대·축소)과 세로 줌(트랙 높이 ↕+ / ↕−)이 이미 검증된 세 함수를 쓰도록 바꿨습니다.

- **가로 확대·축소**: `setZoom()` 이 한계를 `clampZoom()` 으로 맞추고, 스크롤 위치를 `zoomScrollLeft()` 로 옮깁니다. 마우스 아래에 있던 마디가 화면에서 제자리에 남습니다.
- **세로 확대·축소**: `stepAllTrackHeights()` 가 `stepTrackHeight()` 로 다음 단계 높이를 구합니다.

```
Ctrl+휠 (마우스 아래 마디 고정)      VIEW ▸ Zoom In/Out      ↕+ / ↕−
배율 0.25 … 12                      1.25배씩                28·36·44·…·200px
```

2. 코드

가로 줌 — `setZoom`

```
export function setZoom(zoom, anchorX) {
  const c = zoomCtx;
  if (!c) return;
  const head = c.scroll.querySelector('.sq-track__head')?.getBoundingClientRect().width ?? 0;
  const x = anchorX ?? head + (c.scroll.clientWidth - head) / 2;
  const oldBarWidth = barWidth(c.lanes);
  c.zoom = clampZoom(zoom, fitZoom(c));
  applyBarWidth(c);
  updateGrid(c);
  c.scroll.scrollLeft = zoomScrollLeft(c.scroll.scrollLeft, x, head, oldBarWidth, barWidth(c.lanes));
}
```

세로 줌 — `stepAllTrackHeights`

```
function stepAllTrackHeights(dir) {
  const next = stepTrackHeight(baseTrackHeight(heightCtx.lanes), dir);
  if (next != null) setAllTrackHeights(next);
  return next;
}
```

최신 문법으로 쓰면 (공부용)

```
const x = anchorX ?? head + (c.scroll.clientWidth - head) / 2;
```

최신 문법	실제 코드	뜻
`??` (null 병합)	`anchorX ?? …`	여기 코드에는 그대로 남아 있습니다(들어온 값이 없을 때만 기본값)

3. 왜 이렇게

선택	이유
`clampZoom()` 재사용	최소(맞춤/ZOOM_MIN)·최대(ZOOM_MAX) 규칙을 이미 시험했습니다(126). 화면 코드가 같은 식을 두 번 적을 필요가 없습니다
`zoomScrollLeft()` 재사용	"마우스 아래 마디 고정" 계산을 이미 시험했습니다(127). 반올림된 마디 폭을 넘겨 정확히 맞춥니다
나머지 반올림은 `applyBarWidth` 가	마디 폭은 정수 px 이어야 선이 흐려지지 않습니다. 새 폭을 잰 뒤 `zoomScrollLeft` 에 넘깁니다
`stepTrackHeight()` 재사용	단계 목록·0.5px 여유 규칙을 이미 시험했습니다(128). 전에는 화면 함수가 목록을 직접 뒤졌습니다
동작(휠·버튼)은 그대로	규칙만 검증된 함수로 모으고, 이벤트 연결은 손대지 않아 되돌리기 쉽습니다

4. 시험

(1) Node 단위 시험 — 모듈 적재 확인(11개 통과). `clampZoom`·`zoomScrollLeft`·`stepTrackHeight` 는 126~128에서 각각 통과.

(2) 브라우저 확인(Chrome headless + CDP 실제 휠·클릭, 실제 `studioNx.html`) — Ctrl+휠과 ↕ 버튼을 썼습니다.

확인	결과
Ctrl+휠 확대 → 배율 1 → 1.16, 마우스 아래 마디 10.34 → 10.33(제자리)	✓
아주 많이 확대 → 배율 12(`ZOOM_MAX`)에서 멈춤	✓
아주 많이 축소 → 배율 0.25(`ZOOM_MIN`)에서 멈춤	✓
↕+ 클릭 → 트랙 높이 44 → 56, ↕− → 56 → 44 (한 단계씩)	✓

JS 148 — 오디오 클립 파형 배선 (`placeholderPeaks` · `drawClipWaveform` · `drawWaveforms` · `initWaveforms`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 서른네 번째 작업 (FR-210 오디오 클립 파형)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	S1-06 남은 정리 후 사용자 요청 시 커밋·PR 을 진행합니다

1. 하는 일

오디오 클립의 파형 미리보기(캔버스)에 파형을 그립니다. 함수 133~135(`waveformColumns`·`waveformBars`·`drawWaveform`)를 실제 캔버스에 연결하고, 클립이 바뀌면 다시 그리도록 배선했습니다.

아직 실제 샘플 데이터가 없어(S1-10 에서 옵니다) 파형 값은 **샘플 이름으로 만든 자리표시 peaks** 를 씁니다. 화면에서 파형이 보이고, 트리밍·확대에 맞춰 다시 그려지는지 확인할 수 있습니다.

```
① 이름 → 자리표시 peaks (0~1 진폭 512칸)        ② 클립 --o---l → tick
   placeholderPeaks('vox_take2')  →  [0.02, 0.41, …]    offset 0, length 10마디
③ waveformColumns: 클립 폭 칸에 맞게 접기(구간 최대값)
④ waveformBars: 가운데 기준 위·아래 좌표
⑤ drawWaveform: 캔버스에 세로 선으로 그리기
```

2. 코드

자리표시 파형 — `placeholderPeaks`

```
const PEAK_COUNT = 512;

const peaksCache = new Map();

function hashName(name) {
  let h = 2166136261;
  for (let i = 0; i < name.length; i++) { h ^= name.charCodeAt(i); h = Math.imul(h, 16777619); }
  return h >>> 0;
}

function placeholderPeaks(name) {
  const hit = peaksCache.get(name);
  if (hit) return hit;
  let seed = hashName(name) || 1;
  const rand = () => { seed = (Math.imul(seed, 1664525) + 1013904223) >>> 0; return seed / 4294967296;
};
  const peaks = new Array(PEAK_COUNT);
```

```
  for (let i = 0; i < PEAK_COUNT; i++) {
    const p = i / PEAK_COUNT;
    const body = 0.30 + 0.35 * Math.sin(p * Math.PI * 6) * Math.sin(p * Math.PI * 1.7);
    let v = Math.abs(body) * (0.55 + 0.9 * rand());
    if (rand() > 0.86) v = 1;                                    // 가끔 강한 타격
    const fade = Math.min(1, p * 20) * Math.min(1, (1 - p) * 20); // 앞뒤 페이드
    peaks[i] = Math.max(0.02, Math.min(1, v * fade));
  }
  peaksCache.set(name, peaks);
  return peaks;
}
```

클립 하나 그리기 — `drawClipWaveform`

```
function drawClipWaveform(clip, dpr) {
  const canvas = clip.querySelector('.sq-clip__preview');
  if (!canvas) return;
  const w = canvas.clientWidth, h = canvas.clientHeight;
  if (w < 2 || h < 2) return;
  const pw = Math.max(1, Math.round(w * dpr)), ph = Math.max(1, Math.round(h * dpr));
  if (canvas.width !== pw || canvas.height !== ph) { canvas.width = pw; canvas.height = ph; }
  const ctx = canvas.getContext('2d');
  ctx.setTransform(1, 0, 0, 1, 0, 0);
  ctx.clearRect(0, 0, pw, ph);
  ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
  const nameEl = clip.querySelector('.sq-clip__name');
  const name = (nameEl ? nameEl.textContent : '').replace(/^[目→]\s*/, '').trim();
  const lengthTick = barsToTicks(parseFloat(clip.style.getPropertyValue('--l')) || 1);
  const offsetTick = barsToTicks(parseFloat(clip.style.getPropertyValue('--o')) || 0);
  const sourceLengthTick = offsetTick + lengthTick;
  const columns = Math.max(1, Math.min(4000, Math.round(w)));
  const cols = waveformColumns(placeholderPeaks(name), offsetTick, lengthTick, sourceLengthTick, column
s);
  const bars = waveformBars(cols, h);
  ctx.strokeStyle = getComputedStyle(clip).getPropertyValue('--tc').trim() || 'currentColor';
  ctx.lineWidth = 1;
  drawWaveform(ctx, bars, w / columns);
}
```

전체 다시 그리기와 배선 — `drawWaveforms` · `initWaveforms`

```
function drawWaveforms() {
  const c = zoomCtx;
  if (!c) return;
  const dpr = window.devicePixelRatio || 1;
  c.lanes.querySelectorAll('.sq-clip--audio').forEach((clip) => drawClipWaveform(clip, dpr));
}

function initWaveforms({ lanes }) {
  new MutationObserver(requestGrid).observe(lanes, { subtree: true, childList: true, attributes: true, a
ttributeFilter: ['style'] });
  requestGrid();
}
```

격자와 함께 다시 그리기 — `requestGrid` 한 줄 추가

```
function requestGrid() {
  if (gridFrame) return;
  gridFrame = requestAnimationFrame(() => { gridFrame = 0; drawGrid(); drawWaveforms(); });
}
```

CSS — 자리표시 그라디언트 제거 (`studio.css`)

```
.sq-clip--audio .sq-clip__preview { background: none; mask: none; }   /* 파형은 JS 가 캔버스에 그림 (FR-210) */
```

최신 문법으로 쓰면 (공부용)

```
const name = (clip.querySelector('.sq-clip__name')?.textContent ?? '').replace(/^[目⇥]\s*/, '').trim();
```

최신 문법	실제 코드	뜻
`?.` (옵셔널 체이닝)	`nameEl ? nameEl.textContent : ''`	이름 요소가 없으면 빈 글자입니다. 실제 코드는 `if`/삼항으로 풀어 씁니다
`??` (null 병합)	`...	

3. 왜 이렇게

선택	이유
자리표시 peaks (가짜 파형)	실제 샘플 로딩·디코딩은 S1-10 입니다. 그 전에도 배선(캔버스 그리기·다시 그리기)을 화면에서 시험할 수 있습니다
이름을 씨앗으로 (`hashName`)	같은 샘플 이름이면 늘 같은 모양이라 새로 그려도 흔들리지 않고, `vox_take2` 와 `riser_8bar` 는 서로 다르게 보입니다. `peaksCache` 로 한 번만 만듭니다
소스 길이 = `offset + length`	실제 소스 전체 길이를 아직 모릅니다. 자리표시에서는 "앞에서 건너뛴 만큼 + 보이는 만큼"을 소스로 봅니다. 그래서 왼쪽을 트리밍하면 파형이 뒤로 밀려 보입니다(실제와 같은 방향)
칸 수 = 캔버스 CSS 폭 (1칸 ≈ 1px)	파형이 화면 폭에 꽉 차고, 확대하면 캔버스가 넓어져 칸이 늘어 더 촘촘히 보입니다. 상한 4000으로 아주 긴 클립도 감당합니다
dpr 로 캔버스 확대	맥·레티나에서 선이 흐려지지 않도록 CSS 폭 × 화면 배율로 캔버스를 잡고, 그리기는 CSS 좌표로 합니다
`MutationObserver` 로 style 변화 감지	클립을 옮기거나(–s) 길이를 바꾸거나(–l) 트리밍(–o)하면 `style` 속성이 바뀝니다. 그걸 잡아 다시 그립니다. 이벤트마다 다시 그리기 코드를 붙이지 않아 배선이 한 곳에 모입니다
`requestGrid` 에 얹기	격자도 확대·스크롤 때 rAF 로 한 번만 다시 그립니다. 파형도 같은 시점에 그려야 클립 폭과 어긋나지 않고, 그리기 횟수도 한 번으로 묶입니다
캔버스 투명 + CSS 그라디언트 제거	예전엔 CSS 가 흉내 낸 파형이었습니다. 이제 진짜 파형을 캔버스에 그리므로 아래 그라디언트와 겹치지 않게 지웁니다
색은 `--tc` (트랙 색)	클립 색 계열과 맞춥니다. 글자색(`currentColor`)은 어두울 수 있어 트랙 색을 먼저 씁니다

4. 시험

(1) Node 단위 시험 — 모듈 적재 확인(11개 통과). `waveformColumns`·`waveformBars`·`drawWaveform` 은 133~135에서 각각 통과했습니다.

(2) 브라우저 확인(Chrome headless + CDP, 실제 `studioNx.html`) — 캔버스 픽셀을 읽어(투명하지 않은 점 개수·서명) 확인했습니다.

확인	결과
오디오 클립 캔버스에 파형이 그려짐(vox_take2 288px → 3430점)	✓
같은 이름 두 클립(riser_8bar)의 파형 서명이 같음(736431 = 736431)	✓
다른 이름(vox_take2 vs riser_8bar)은 서명이 다름	✓
MIDI 클립 캔버스는 그리지 않음(0점) — CSS 그라디언트 유지	✓
`--1` 을 10 → 6 으로 바꾸면 다시 그려짐(288 → 172px, 3430 → 2264점)	✓
Ctrl+휠 확대 시 캔버스가 넓어지며 다시 그려짐(288 → 848px, 3430 → 8380점)	✓

스크린샷: `docs/ai-notes/images/javascript_s1-06/s1-06_audio-waveform.png`